Practical – 1

Aim : Import the data warehouse data in Microsoft Excel and create the Pivot table and Pivot Chart.

Pivot Table : A Pivot Table is a powerful Excel tool used to summarize, analyze, and organize large amounts of data**.**

Pivot Chart : A Pivot Table is a powerful Excel tool used to summarize, analyze, and organize large amounts of data.

Steps To Create a **Pivot Table and Pivot Chart :**

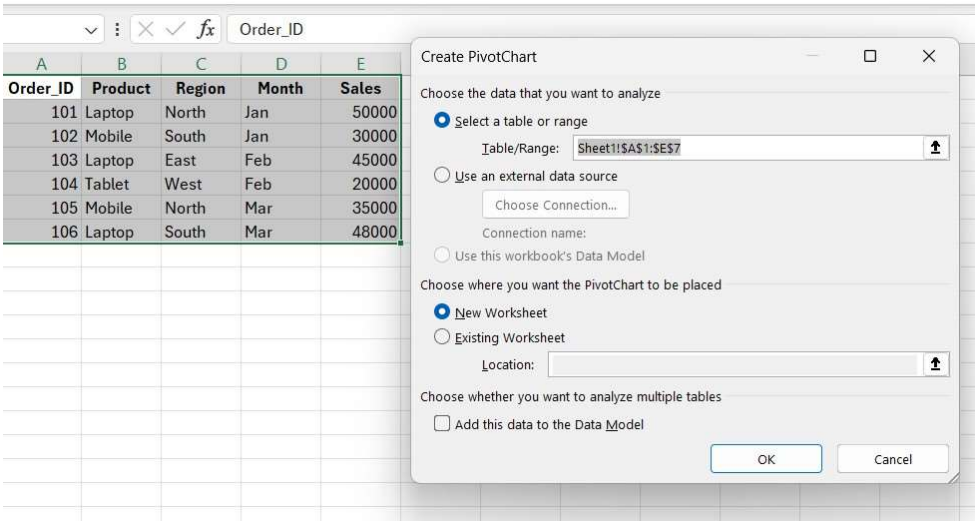
Step 1: Import Data into Excel

1. Open Microsoft Excel and Create a new worksheet.
2. Enter the above data in rows and columns and Save the file .

Order_id	Product	Region	Month	Sale
101	Laptop	North	Jan	50000
102	Mobile	South	Jan	30000
103	Laptop	East	Jan	45000
104	Tablet	West	Feb	20000
105	Mobile	North	Mar	35000
106	Laptop	South	Mar	48000

Step 2: Create Pivot Table

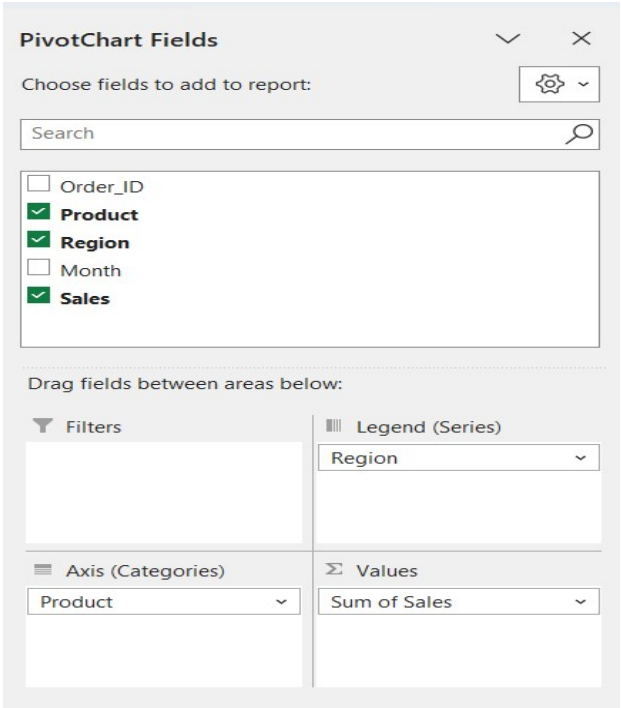
1. **Select the complete data and Click Insert → PivotTable .**
2. **Choose From Table/Range.**
3. **Select New Worksheet and Click OK .**



Step 3: Design Pivot Table

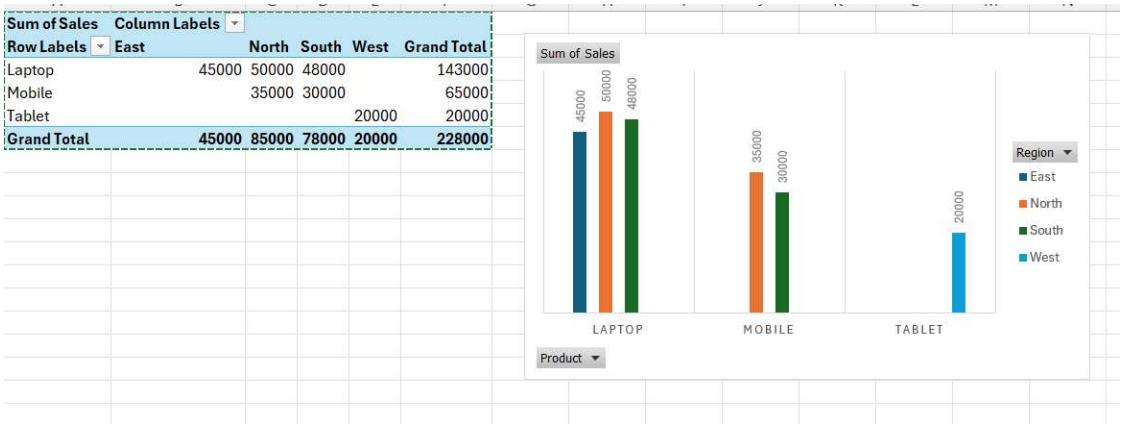
- Drag Product → Rows
- Drag Region → Columns

- Drag Sales → Values (Sum of Sales)



Step 4: Create Pivot Chart

1. Click anywhere on the Pivot Table
2. Go to Insert → PivotChart
3. Click OK



Sum of Sales	Column Labels				
Row Labels	East	North	South	West	Grand Total
Laptop	45000	50000	48000		143000
Mobile		35000	30000		65000
Tablet				20000	20000
Grand Total	45000	85000	78000	20000	228000

Practical – 2

Aim : Apply the what – if Analysis for data visualization. Design and generate necessary reports based on the data warehouse data. Use Excel.

What-If Analysis : is an Excel feature used to **change input values and observe how those changes affect the final result**. It helps in **decision making and forecasting**. It contains three method :

a. **Scenario Manager** : Scenario Manager is an Excel What-If Analysis tool used to **compare different sets of input values** and see how they affect the final result.
b. **Goal Seek** : Goal Seek is an Excel tool used to **find the required input value** needed to achieve a **specific desired output**.
c. **Data Table** : Data Table is an Excel What-If Analysis tool used to **display multiple results** by changing **one or two input values** at a time.

Step 1: Create CSV File

1. Open **Microsoft Excel**.
2. Create a table with the following data:

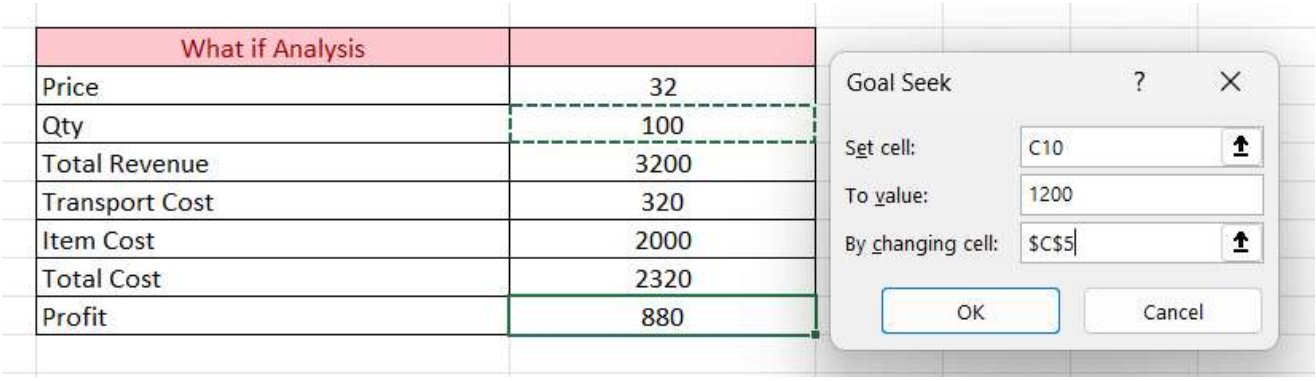
What if Analysis	
Price	32
Qty	100
Total Revenue	3200
Transport Cost	320
Item Cost	2000
Total Cost	2320
Profit	880

Step 2: Apply What-If Analysis (Goal Seek).

1. Go to **Data → What-If Analysis → Goal Seek**
2. Set:

- **Set cell:** Profit cell → **To value:** 1200 (desired profit) →**By changing cell:** Quantity

3. Click **OK**

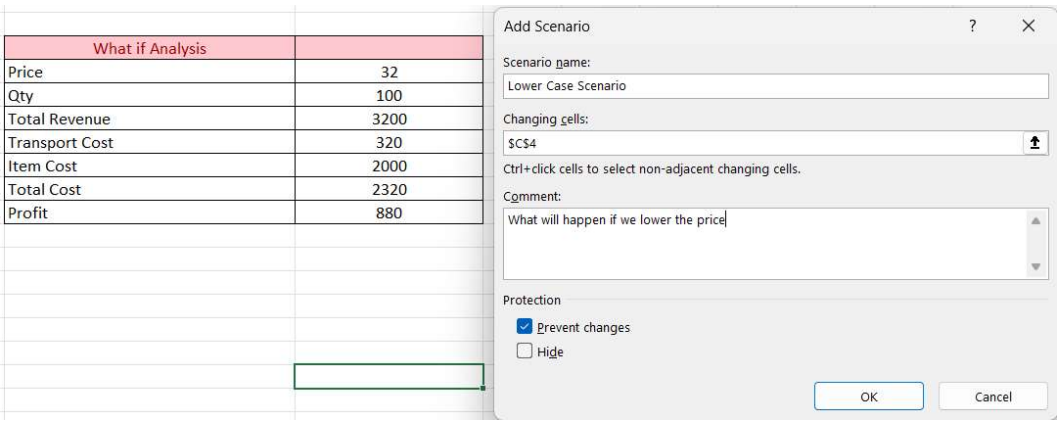


- Excel automatically adjusts the quantity to achieve the target profit.

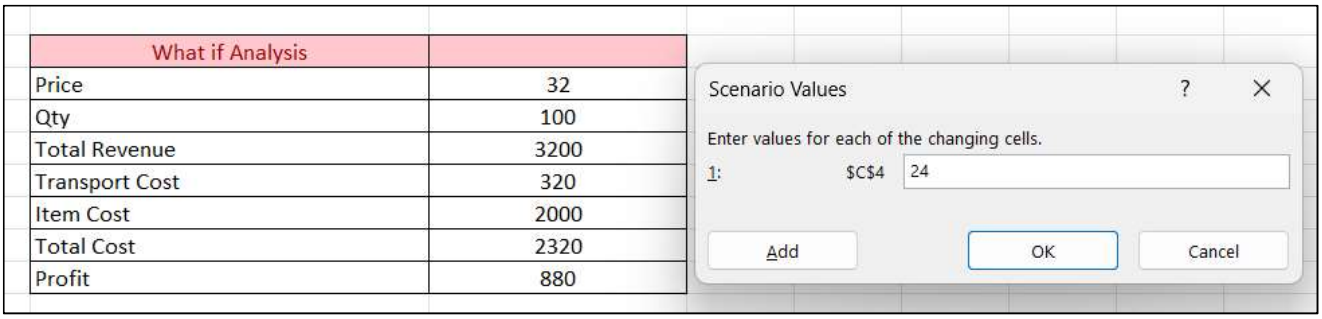
What if Analysis	
Price	32
Qty	136.3636364
Total Revenue	4363.636364
Transport Cost	436.3636364
Item Cost	2727.272727
Total Cost	3163.636364
Profit	1200

Step 4: Apply What-If Analysis (Scenario Manager)

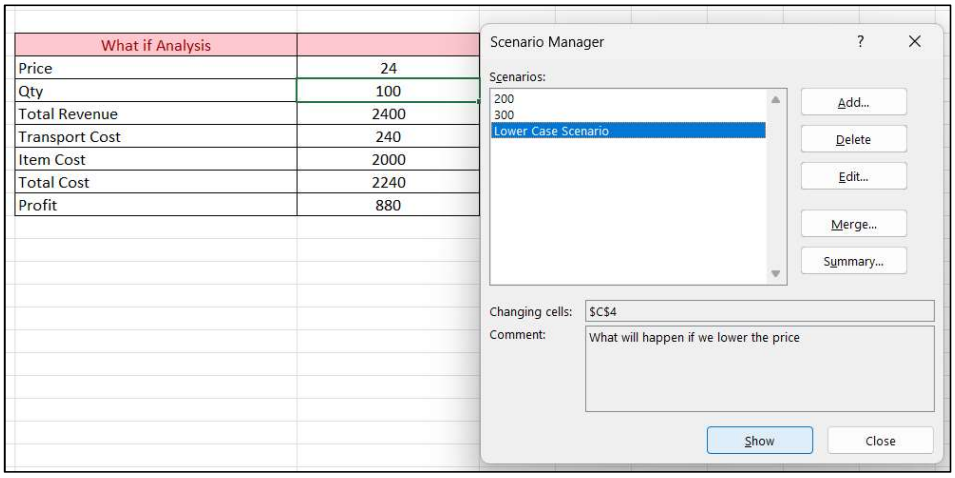
1. Go to **Data → What-If Analysis → Scenario Manager**
2. Click **Add Scenario** :



- if we lower the price to 32 to 24 will see how It will affect the Total Revenue and Total Cost



- Click Add And Select Low case Scenarios and Click Show



Practical 3

Aim**:** To perform the data classification using classification algorithm using Python.

Definition

Data Classification**:** is the process of dividing data into predefined / specific groups based on its features using a classification algorithm. (it's a supervised learning technique)

Decision Tree Classification Algorithm**:** is a classification algorithm that splits data into branches using simple conditions to decide which class or group the data belongs to.

Tools Required

- Python
- Pandas Library
- Matplotlib Library

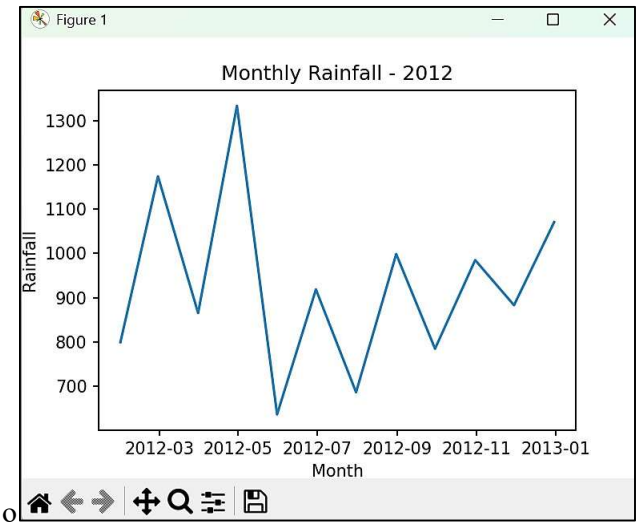
Step 1: Open the Python environment.
Step 2: Import required libraries (pandas and matplotlib).
Step 3: Create the rainfall dataset and generate the monthly date range.
Step 4: Convert the data into a time series and display it.
Step 5: Plot the rainfall data using a line graph with appropriate labels.
Step 6: Save and display the graph.

---Python Code---

```
import pandas as pd
import matplotlib.pyplot as plt

# Rainfall data
rainfall = [799, 1174.8, 865.1, 1334.6, 635.4, 918.5,
            685.5, 998.6, 784.2, 985, 882.8, 1071]

# Create date range starting Jan 2012
dates = pd.date_range(start="2012-01-01", periods=12, freq="ME")

# Convert into time series
rainfall_series = pd.Series(rainfall, index=dates)

# Print time series
print(rainfall_series)

plt.plot(rainfall_series)
plt.title("Monthly Rainfall - 2012")
plt.xlabel("Month")
plt.ylabel("Rainfall")
plt.savefig("rainfall.png")
plt.show()
```

```
File  Edit  Format  Run  Options  Window  Help
import pandas as pd
import matplotlib.pyplot as plt

# Rainfall data (same as R vector)
rainfall = [799, 1174.8, 865.1, 1334.6, 635.4, 918.5,
            685.5, 998.6, 784.2, 985, 882.8, 1071]

## Create date range starting Jan 2012 (Use ME instead of M)
dates = pd.date_range(start="2012-01-01", periods=12, freq="ME")

# Convert into time series
rainfall_series = pd.Series(rainfall, index=dates)

# Print time series
print(rainfall_series)

plt.plot(rainfall_series)
plt.title("Monthly Rainfall - 2012")
plt.xlabel("Month")
plt.ylabel("Rainfall")
plt.savefig("rainfall.png")
plt.show()
```

---OUTPUT---




```
= RESTART: C:\sadiya shaikh\BI\
2012-01-31     799.0
2012-02-29    1174.8
2012-03-31     865.1
2012-04-30    1334.6
2012-05-31     635.4
2012-06-30     918.5
2012-07-31     685.5
2012-08-31     998.6
2012-09-30     784.2
2012-10-31     985.0
2012-11-30     882.8
2012-12-31    1071.0
Freq: ME, dtype: float64
```

Result

The rainfall time series data was successfully generated and visualized using Python.

Practical 4

Aim**:** To perform the data clustering using clustering algorithm using Python.

Definition:

Data clustering**:** is a method of grouping similar data points together based on their features, without using predefined labels. (it's an unsupervised learning technique).

K-Means**:** is a clustering algorithm that divides data into a fixed number of groups (K) by grouping nearby data points together based on distance._

Tools Required

- Python
- Scikit-learn
- Pandas
- Matplotlib

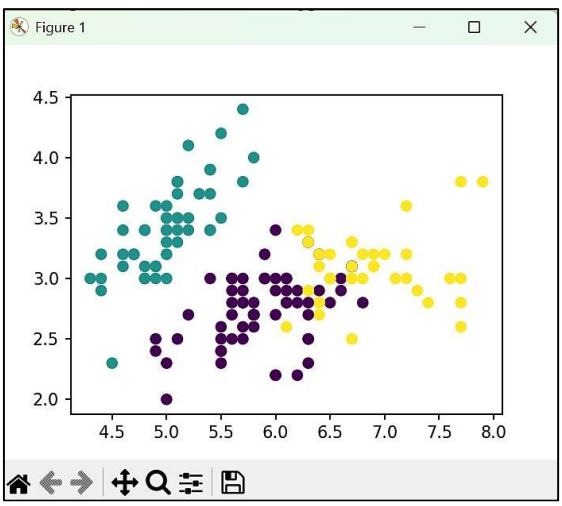
Step 1: Open the Python environment.
Step 2: Import required libraries and load the Iris dataset.
Step 3: Select the feature variables for clustering.
Step 4: Apply the K-Means algorithm with 3 clusters.
Step 5: Fit the model and obtain cluster labels.
Step 6: Display cluster sizes, cluster means, clustering vector, and within-cluster sum of squares.
Step 7: Compare actual species with predicted clusters using a cross-tabulation table.
Step 8: Visualize the clusters using a scatter plot.

---Python Code---

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import pandas as pd
import matplotlib.pyplot as plt

# Load iris dataset
iris = load_iris()
X = iris.data

# Apply K-means clustering
kc = KMeans(n_clusters=3, random_state=0).fit(X)

# Cluster sizes
print("Cluster sizes:")
print(pd.Series(kc.labels_).value_counts().sort_index())
print()

# Cluster means
print("Cluster means:")
print(kc.cluster_centers_)
print()
```

```
# Clustering vector
print("Clustering vector:")
print(kc.labels_)
print()

# Within cluster sum of squares
print("Within cluster sum of squares:")
print(kc.inertia_)
print()

# Species vs cluster table
print("Species vs Cluster:")
print(pd.crosstab(iris.target, kc.labels_))
print()

# Plot clusters
plt.scatter(X[:,0], X[:,1], c=kc.labels_)
plt.show()
```

---OUTPUT---




```
Cluster sizes:
0    61
1    50
2    39
Name: count, dtype: int64

Cluster means:
[[5.88360656 2.74098361 4.38852459 1.43442623]
 [5.006      3.428      1.462      0.246     ]
 [6.85384615 3.07692308 5.71538462 2.05384615]]

Clustering vector:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 2 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 2 2 2 2 0 2 2 2 2
 2 2 0 0 2 2 2 2 0 2 0 2 0 2 2 0 0 2 2 2 2 2 0 2 2 2 2 0 2 2 2 0 2 2 2 0 2
 2 0]

Within cluster sum of squares:
78.8556658259773

Species vs Cluster:
col_0   0   1   2
row_0
0       0  50   0
1      47   0   3
2      14   0  36
```

Result

The Iris dataset was successfully clustered into three groups using the K-Means clustering algorithm. The cluster sizes, cluster centers, and within-cluster sum of squares were obtained. A comparison between actual species and predicted clusters was performed, and the clusters were visualized using a scatter plot.

Practical-5

Aim: Perform the Linear regression on the given data warehouse data using R/Python.

Step 1: Open Python Environment.

Step 2: Install Required Libraries(pandas matplotlib scikit-learn)

Step 3: Create Dataset Define x (input) and y (output) values.

Step 4: Train Model Create LinearRegression model and fit it using model.fit(x, y).

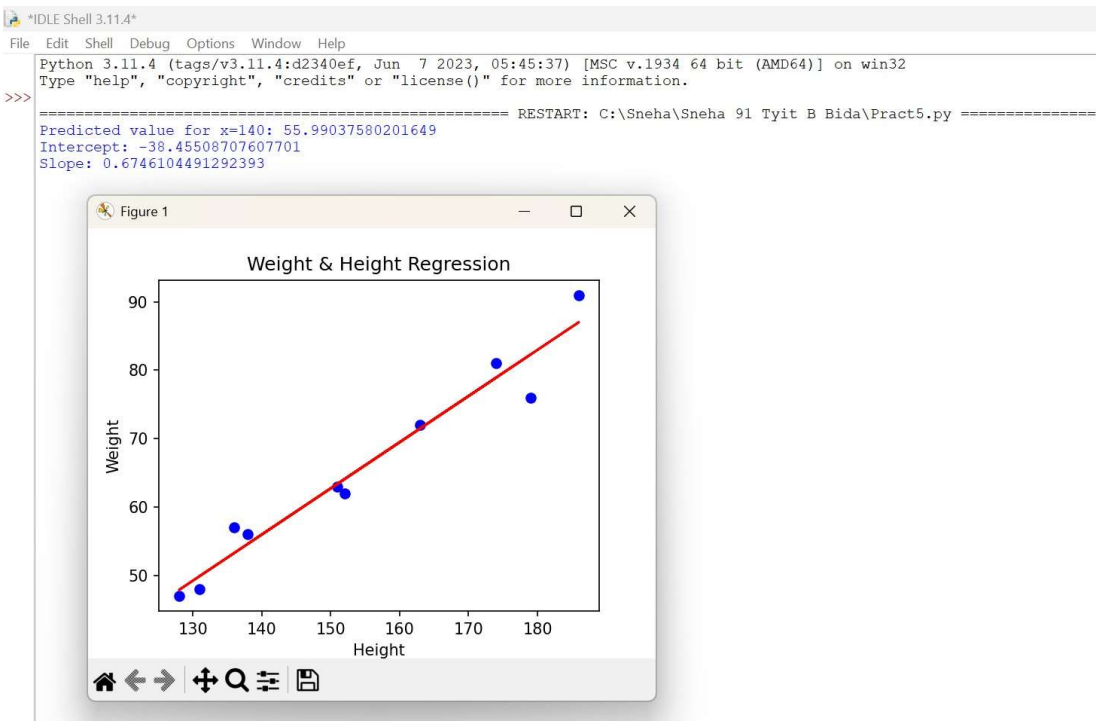
Step 5: Predict & Plot Predict value for x=140 and plot regression graph.

--Python Code--

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Create data
x = np.array([151, 174, 138, 186, 128, 136, 179, 163, 152, 131]).reshape(-1, 1)
y = np.array([63, 81, 56, 91, 47, 57, 76, 72, 62, 48])

# Build linear regression model
model = LinearRegression()
model.fit(x, y)

# Predict value for x = 140
prediction = model.predict([[140]])
print("Predicted value for x=140:", prediction[0])

# Print model parameters
print("Intercept:", model.intercept_)
print("Slope:", model.coef_[0])
```

```
# Plot graph
plt.scatter(x, y, color='blue')
plt.plot(x, model.predict(x), color='red')
plt.title("Weight & Height Regression")
plt.xlabel("Height")
plt.ylabel("Weight")
plt.savefig("linearreg.png")
plt.show()
```

--Output--



Practical-6

Aim: Perform the logistic regression on the given data warehouse data using R/Python.

Step 1: Open Python Environment.

Step 2: Import Libraries & Load Data(Import numpy, pandas, matplotlib, sklearn, and load mtcars dataset using pydataset.)

Step 3: Prepare Data Select features (cyl, hp, wt) as X and target (am) as **y`.

Step 4: Train Logistic Regression Model Create model using LogisticRegression() and fit using model.fit(X, y).

Step 5: Predict & Evaluate. Predict output, print coefficients & classification report, and plot graph using wt.

--Python Code--

```
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import statsmodels.api as sm
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Load mtcars dataset (manually create it if not available)
# You can install pydataset: pip install pydataset
from pydataset import data
mtcars = data('mtcars')
```

```
# Select required columns
input_data = mtcars[['am', 'cyl', 'hp', 'wt']]
print(input_data.head())

# Define independent and dependent variables
X = input_data[['cyl', 'hp', 'wt']]
y = input_data['am']

# Build logistic regression model
log_model = LogisticRegression()
log_model.fit(X, y)

# Print coefficients
print("Intercept:", log_model.intercept_)
print("Coefficients:", log_model.coef_)

# Predict
y_pred = log_model.predict(X)

# Classification report
print(classification_report(y, y_pred))

# Plot using wt only
X1 = input_data[['wt']]
model2 = LogisticRegression()
model2.fit(X1, y)

plt.scatter(X1, y)
x_test = np.linspace(X1.min(), X1.max(), 100).reshape(-1,1)
plt.plot(x_test, model2.predict_proba(x_test)[:,1])
plt.show()
```

--Output--

```
=======
                am  cyl   hp    wt
Mazda RX4          1    6  110  2.620
Mazda RX4 Wag      1    6  110  2.875
Datsun 710         1    4   93  2.320
Hornet 4 Drive     0    6  110  3.215
Hornet Sportabout  0    8  175  3.440
Intercept: [6.97297795]
Coefficients: [[-0.71109921  0.02113021 -1.9902589 ]]
              precision    recall  f1-score   support

           0       0.90      0.95      0.92        19
           1       0.92      0.85      0.88        13

    accuracy                           0.91        32
   macro avg       0.91      0.90      0.90        32
weighted avg       0.91      0.91      0.91        32
```

Practical-7

Aim: Write a Python program to read data from a CSV file, perform simple data analytics, and generate basic insights using **Pandas** library.

Pandas is a powerful Python library used for data analysis and data manipulation.
It helps in reading data from CSV files, analyzing it, and generating useful insights such as total, average, maximum, and minimum values.

Step 1: Create CSV File

3. Open **Microsoft Excel**.
4. Create a table with the following data:

	A	B
1	Category	Sales
2	Electronics	5000
3	Clothing	3000
4	Grocery	2000
5		

3. Save the file as **sales.csv**.

Step 2: Install Pandas Library

If Pandas is not installed, run the following command inside python folder->scripts->(cmd):

pip install pandas

```
\Python311\Scripts>pip install pandas
```

Step 3: Write Python Program

import pandas as pd

```
# Read CSV file
data = pd.read_csv("sales.csv")

print("Data from CSV file:")
print(data)

# Basic information
print("\nBasic Information:")
print(data.info())

# Statistical summary
print("\nStatistical Summary:")
print(data.describe())

# Calculated insights
total_sales = data["Sales"].sum()
average_sales = data["Sales"].mean()
highest_sales = data["Sales"].max()
lowest_sales = data["Sales"].min()

print("\nInsights:")
print("Total Sales:", total_sales)
print("Average Sales:", average_sales)
print("Highest Sales:", highest_sales)
print("Lowest Sales:", lowest_sales)

# Category-wise sales
category_sales = data.groupby("Category")["Sales"].sum()

print("\nCategory-wise Sales:")
print(category_sales)
```

Step 4: Output / Result

- Data from CSV file is displayed.
- Basic information of data is shown.
- Statistical summary is generated.
- Total, average, highest, and lowest sales values are calculated.
- Category-wise sales analysis is displayed.

```
Data from CSV file:
      Category  Sales
0  Electronics   5000
1     Clothing   3000
2      Grocery   2000

Basic Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Category  3 non-null      object
 1   Sales     3 non-null      int64
dtypes: int64(1), object(1)
memory usage: 176.0+ bytes
None

Statistical Summary:
             Sales
count     3.000000
mean   3333.333333
std    1527.525232
min    2000.000000
25%    2500.000000
50%    3000.000000
75%    4000.000000
max    5000.000000
```

```
Insights:
Total Sales: 10000
Average Sales: 3333.3333333333335
Highest Sales: 5000
Lowest Sales: 2000

Category-wise Sales:
Category
Clothing       3000
Electronics    5000
Grocery        2000
```

Practical – 8

Aim : Perform data visualization using Python on any sales data**.**

To perform data visualization using Python we need to import certain libraries are as follows:

a. **Pandas**: Pandas is a Python library used for **data manipulation and analysis**. It helps in **reading, cleaning, organizing, and analyzing structured data** such as tables from CSV, Excel, or databases.
b. **Matplotlib :** Matplotlib is a Python library used for creating static, animated, and interactive visualizations. It is mainly used to plot graphs and charts like line graphs, bar charts, histograms, and pie charts.
c. **Seaborn :** Seaborn is a Python data visualization library built on top of Matplotlib. It is used to create attractive and informative statistical graphics with less code and better default styling**.**

Step 1: Create CSV File

1. Open **Microsoft Excel**.
2. Create a table with the following data:

Month	Product	Sales
Jan	Pen	1000
Jan	Pencil	800
Feb	Pen	1200
Feb	Pencil	900
Mar	Pen	1500
Mar	Penicil	1100

3. Save the file as **sales.csv**.

Step 2 : Import Library .

If its not installed, run the following command inside python folder->scripts->(cmd):

```
C:\Program Files\Python313\Scripts>pip install pandas
```

```
C:\Program Files\Python313\Scripts>pip install matplotlib
```

```
C:\Program Files\Python313\Scripts>pip install seaborn
```

Step 2 : Write Python Program.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Create Sample Sales Data
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Product_A": [12000, 15000, 17000, 16000, 18000, 20000],
    "Product_B": [10000, 12000, 14000, 13000, 15000, 17000],
    "Product_C": [8000, 9000, 10000, 11000, 12000, 14000]
}
df = pd.DataFrame(data)
# Line Plot – Monthly Sales
plt.figure(figsize=(8, 5))
plt.plot(df["Month"], df["Product_A"], marker='o', label="Product A")
plt.plot(df["Month"], df["Product_B"], marker='o', label="Product B")
plt.plot(df["Month"], df["Product_C"], marker='o', label="Product C")
plt.title("Monthly Sales Trend")
```

```
plt.xlabel("Month")
plt.ylabel("Sales Amount")
plt.legend()
plt.grid(True)
plt.show()

# Bar Chart – Sales Comparison
df.set_index("Month").plot(kind="bar", figsize=(8, 5))
plt.title("Sales Comparison by Product")
plt.xlabel("Month")
plt.ylabel("Sales Amount")
plt.show()
# Pie Chart – Total Sales Share
total_sales = df[["Product_A", "Product_B", "Product_C"]].sum()
plt.figure(figsize=(6, 6))
plt.pie(total_sales, labels=total_sales.index, autopct='%1.1f%%')
plt.title("Total Sales Distribution")
plt.show()
# Heatmap – Sales Intensity
plt.figure(figsize=(8, 5))
sns.heatmap(df.drop("Month", axis=1), annot=True, cmap="YlGnBu")
plt.title("Sales Heatmap")
plt.show()
```

OUTPUT:

Figure 1
Monthly Sales Trend
Product A
Product B
Product C
20000
18000
16000
14000
12000
10000
8000
Jan
Feb
Mar
Apr
May
Jun
(x, y) = (Jan, 1.647e+04)

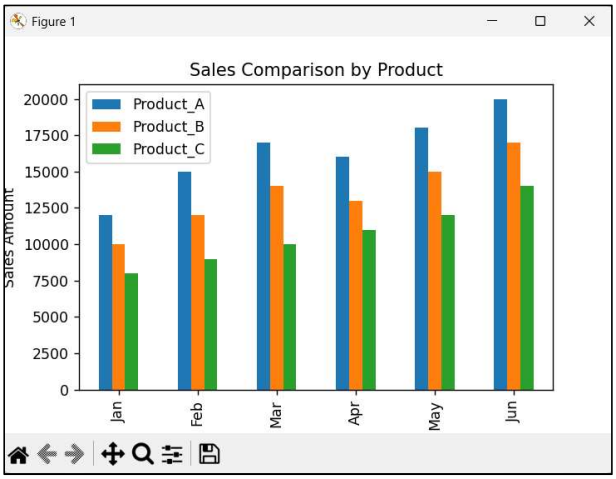


Figure 1
Sales Comparison by Product
Product_A
Product_B
Product_C
Sales Amount
20000
17500
15000
12500
10000
7500
5000
2500
0
Jan
Feb
Mar
Apr
May
Jun

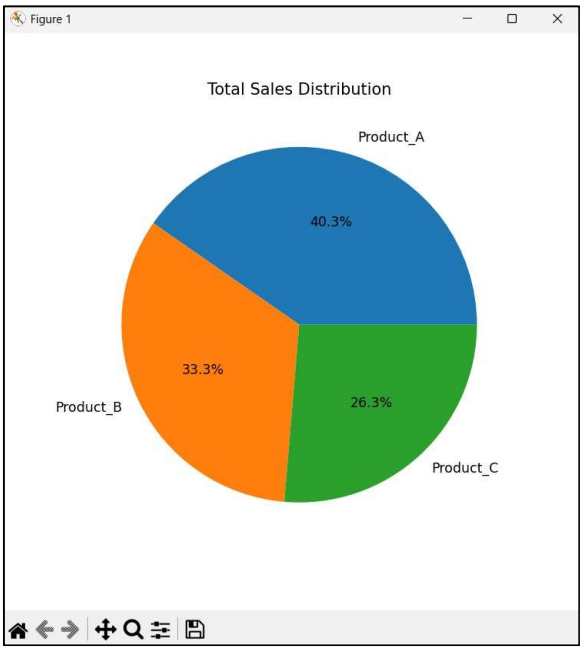


Figure 1
Total Sales Distribution
Product_A
40.3%
33.3%
Product_B
26.3%
Product_C

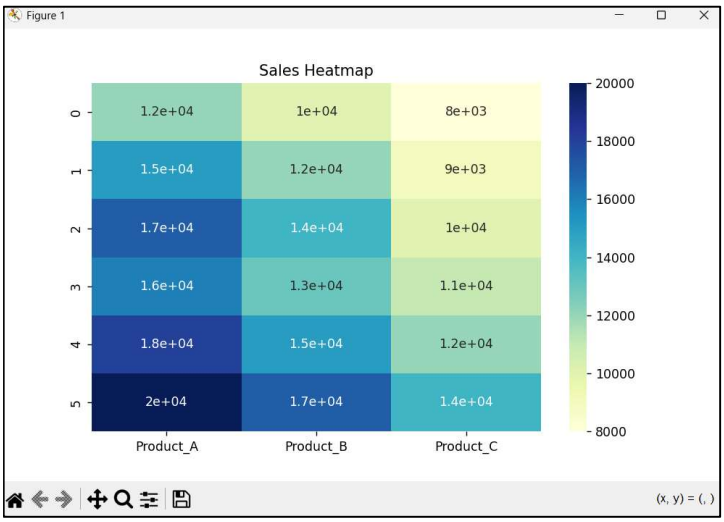


Figure 1
Sales Heatmap
0
1.2e+04
1e+04
8e+03
1
1.5e+04
1.2e+04
9e+03
2
1.7e+04
1.4e+04
1e+04
3
1.6e+04
1.3e+04
1.1e+04
4
1.8e+04
1.5e+04
1.2e+04
5
2e+04
1.7e+04
1.4e+04
Product_A
Product_B
Product_C
20000
18000
16000
14000
12000
10000
8000
(x, y) = (,)

Practical-8b

Aim : Perform data visualization using PowerBI on any sales data.

8-B) Data Visualization using Power BI

Aim**:** Perform data visualization using Python on any sales data.

Definition

Power BI**:** Power BI is a business intelligence tool by Microsoft used to connect to data sources, create interactive reports and dashboards, and visualize data for easy analysis and decision-making.

Tools Required:

• Power BI Desktop

• Sales Dataset (Excel/CSV file)

Dataset Used:

Month	Product	Sales
Jan	Pen	1000
Jan	Pencil	800
Feb	Pen	1200
Feb	Pencil	900
Mar	Pen	1500
Mar	Pencil	1100

(Save this file as **sales.xlsx** or **sales.csv**.)

Step 1: Open Power BI Desktop. → Select Blank Report

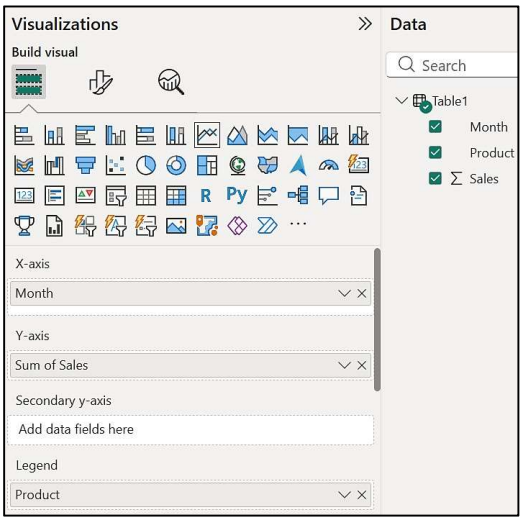
Step 2: Go to the **Home tab** on the ribbon toolbar and click **Get Data**.

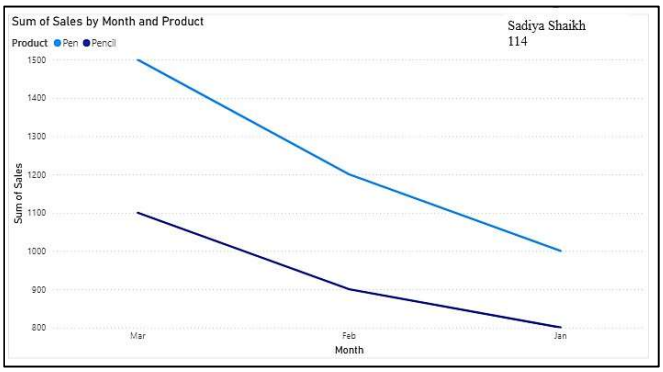
Step 3: Select **Excel** or **Text/CSV**, browse and select the sales dataset file, then click **Load**.

Month	Product	Sum of Sales
Feb	Pen	1200
Feb	Pencil	900
Jan	Pen	1000
Jan	Pencil	800
Mar	Pen	1500
Mar	Pencil	1100
Total		**6500**

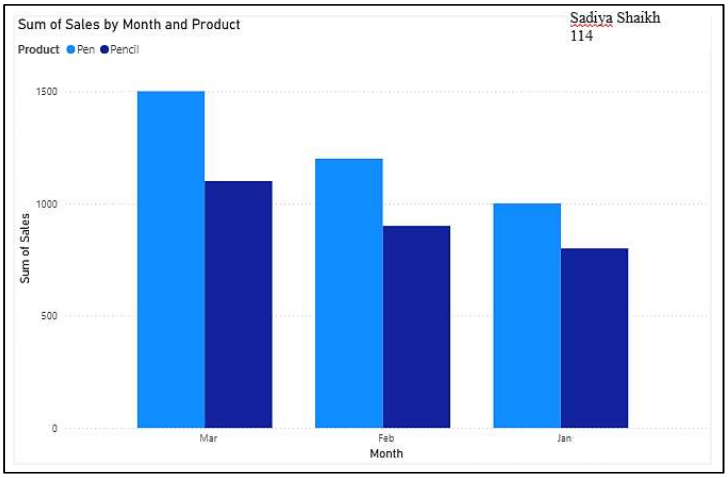
Step 4: Create a **Line Chart**:

- **X-axis** → Drag **Month**
- **Y-axis** → Drag **Sales**
- **Legend** → Drag **Product**

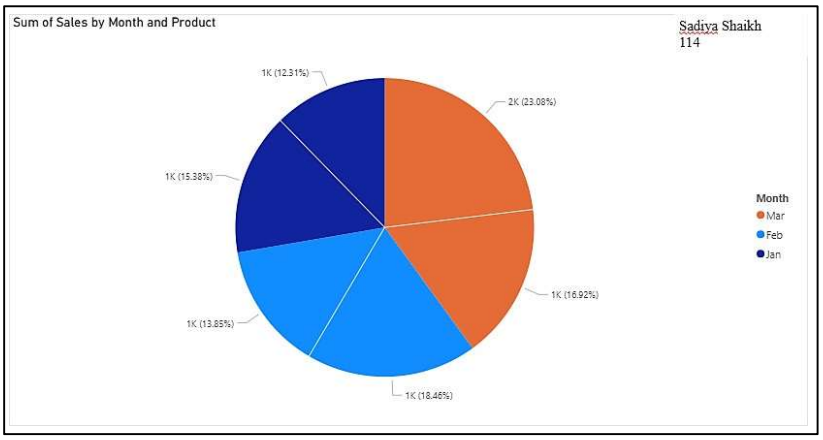




Step 5: Create a **Clustered Bar Chart** to compare product sales across months.



Step 6: Create a **Pie Chart** to show total sales distribution by product.



Step 7: Create a **Matrix or Heatmap-style table** using conditional formatting to represent sales intensity.

<u>Steps to Create a Heatmap Using Matrix:</u>

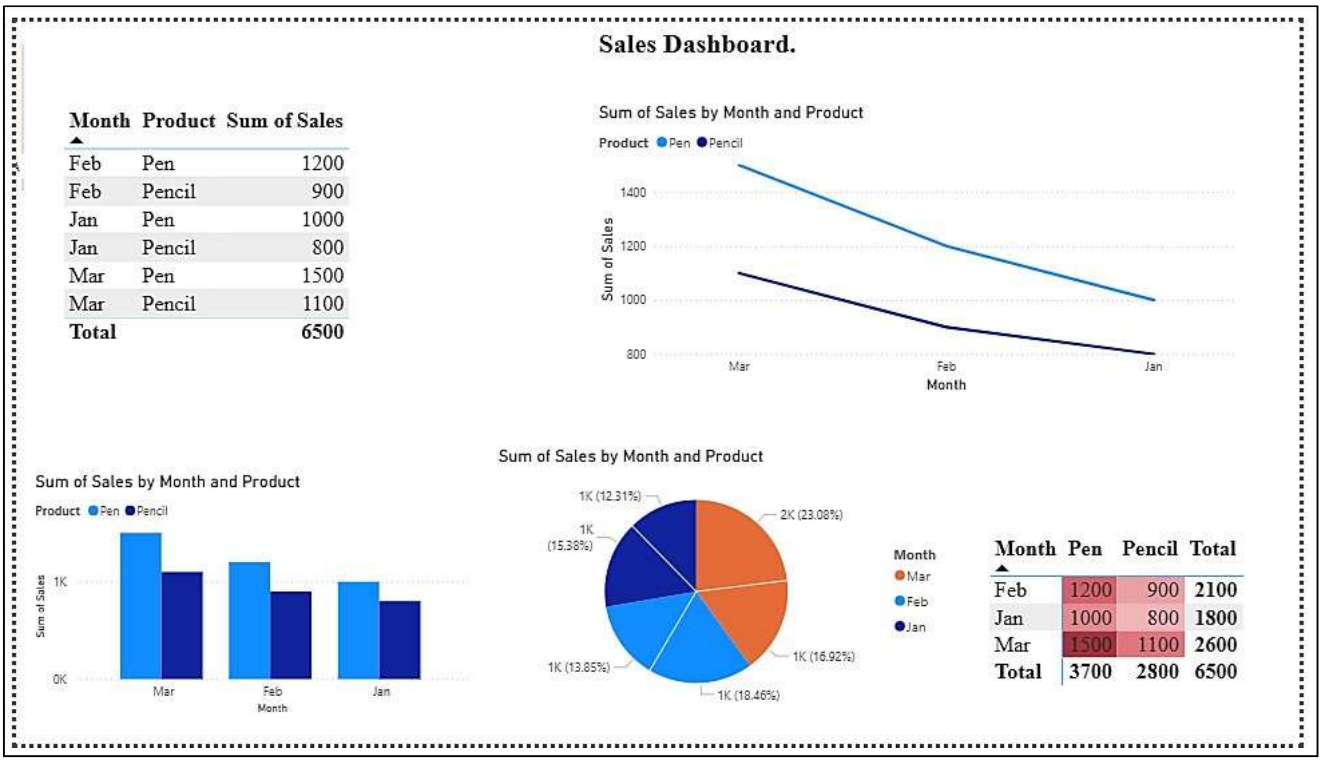
- Select **Matrix** visual.
- Drag:
  - **Rows** → Month
  - **Columns** → Product
  - **Values** → Sales

- Click dropdown beside **Sales → Conditional Formatting → Background Color →** Apply color scale.

Month	Pen	Pencil	Total
Feb	1200	900	2100
Jan	1000	800	1800
Mar	1500	1100	2600
Total	**3700**	**2800**	**6500**

(Darker color represents **higher sales** and lighter color represents **lower sales**.)

Step 8: Arrange all visuals properly to create a simple sales dashboard.



Step 9: Save the Power BI report file (.pbix).

Result:

- Sales data was successfully imported into Power BI.
- Line Chart showed monthly sales trend.
- Bar Chart compared product sales month-wise.
- Pie Chart displayed total sales distribution.
- Matrix visual represented sales intensity.
- A simple sales dashboard was created successfully.

Practical 9

Aim: Create the Data staging area for the selected database using SQL.

STAGING AREA

It is the storage for the sources data used before the ETL process. The source data is preloaded into the staging area which is a separate database with a similar table structure to that of the original data source.

PREPARING THE STAGING AREA FOR THE DATA WAREHOUSE

1. Download or obtain the source data set in CSV format.

2. If the dataset is in the form of a denormalized table you can remove any unwanted columns or rows if applicable. It will speed up the load and also simplify the ETL process later.

3. Launch the SQL Server Management Studio and connect to the desired database engine. Make sure you have an account with privileges to create databases when connecting.

4. Navigate to the Object Explorer and select Databases, right click and Select New Database...and create the database as required to store the staging data.

5. Select the newly created database, right click and select New Query and create the required table schema to load the source data.

To create a single staging table called county_stage using the below script and execute it by pressing F5 key.
create table county_stage(id int state varchar (30), countyname varchar(100));

1. After the staging area database was created the loading of the data needs to be done. For this the Import function of the SQL Server Management Studio was used. Following are the steps.

2. Right click on the created staging database, select Tasks and click on Import Data… Click on Next.

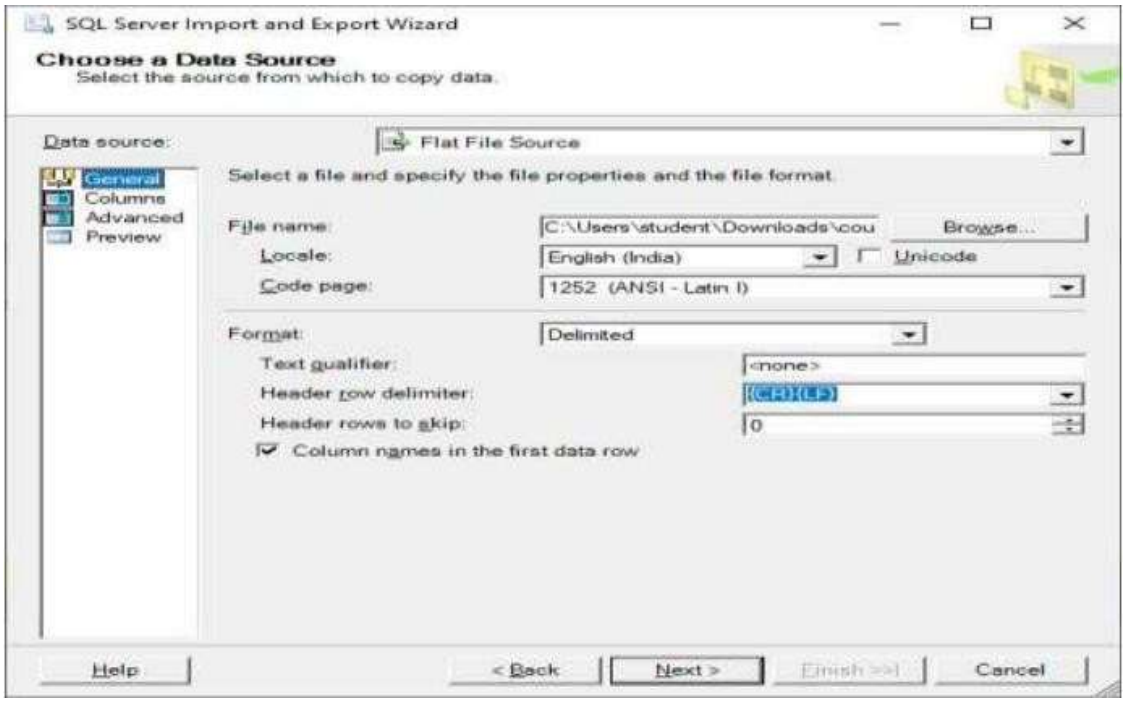
In Choose data source as Flat File Source and browse your CSV file and check the columns in first data row.

Then click on Columns tab and set the proper delimiters and check the columns tab to view if the configuration is working if not set the proper delimiters to get the desired column and row organization.

Next move to Advanced Tab and configure properties of each column such as for ID set data type as four byte signed integer, State set data type as string and OutputColumn Width as 30 and for County Name type as string and OutputColumn Width as 100.

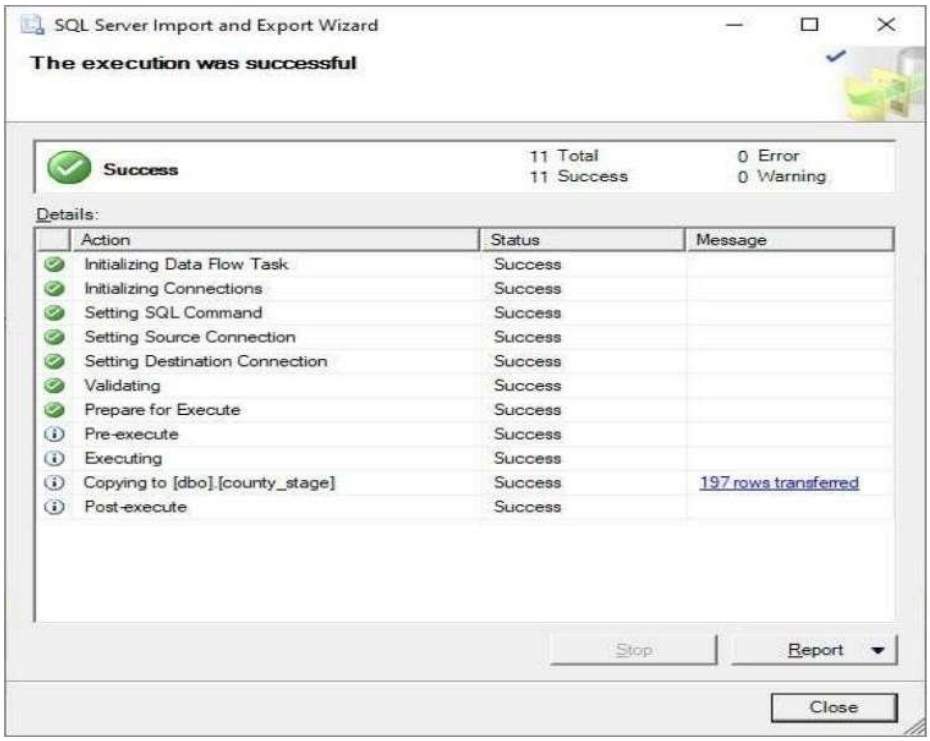
Move to the Preview tab to get a limited preview of the data to be important under given columns, Click Next.

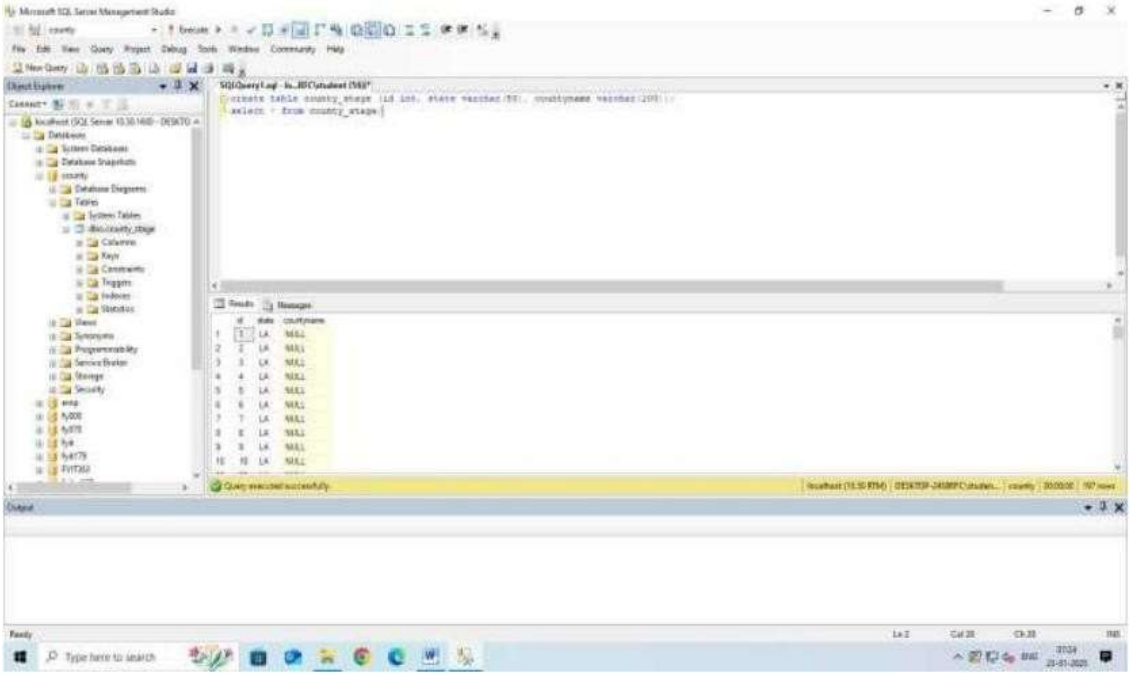
On the Choose a Destination page select appropriate Destination Server Name(localhost) and provide a require authentication to connect (if required) and you will be able to select the staging database created above the destination Database(County) Click Next.



On the Select Source Tables and Views page, map the source to the destination table(s) as required. Click Next.

On the Save and Run Package page select Run Immediately to import the data after configuring the wizard. If you need to connect with a SSIS package you can configure it here. Click Next.




Microsoft SQL Server Management Studio
create table country_stage (id int, state varchar(50), countryname varchar(200));
select * from country_stage
Query executed successfully.

Practical 10

Aim: Create the cube with suitable dimension and fact tables based on ROLAP, MOLAP and HOLAP model.

Creating Data Warehouse

Follow the given steps to run the query in SSMS (SQL Server Management Studio)

1. Open SQL Server Management Studio 2008

2. Connect Database Engine

3. Open New Query Editor

4. Open SQL Script in query editor

5. To run the given SQL Script, press F5

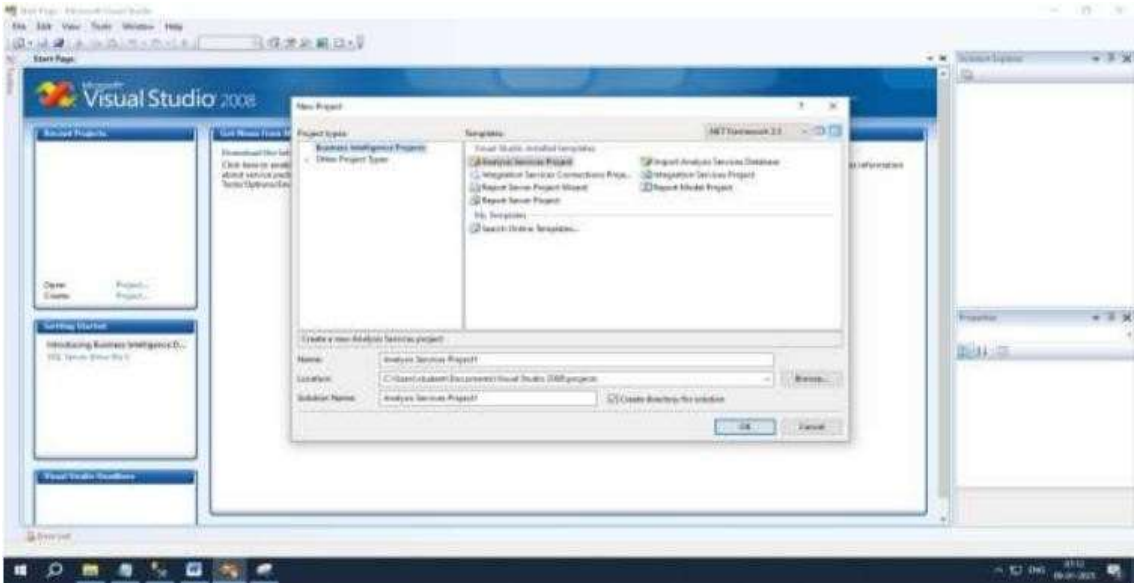
6. It will create and populate “Sales_DW” database on your SQL Server

Developing an OLAP Cube

For creation of OLAP Cube in Microsoft BIDS Environment, follow the 10 easy steps given below.

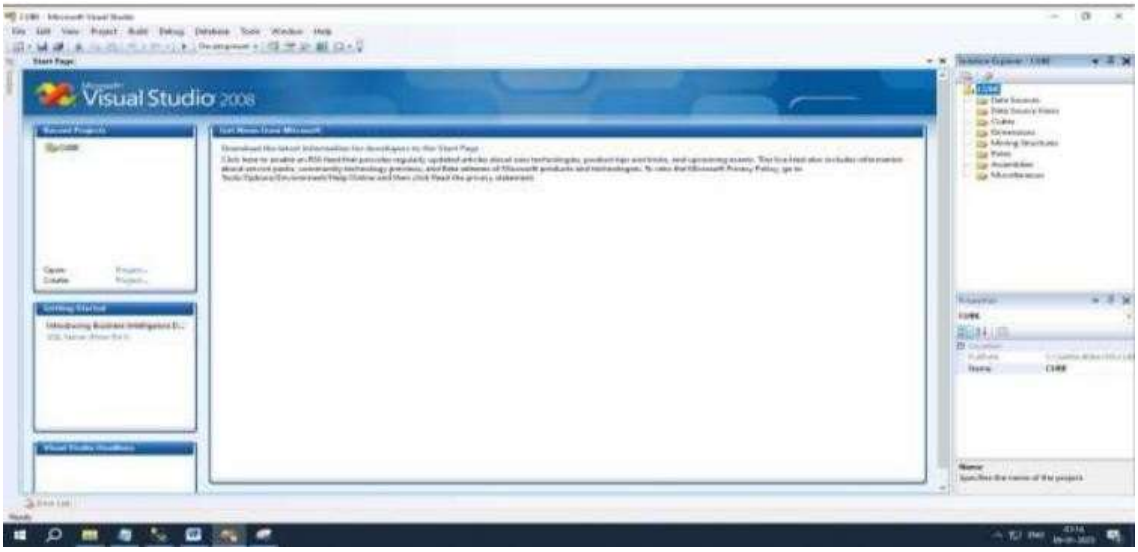
Step 1: Start BIDS Environment. Click on Start ->Microsoft SQL Server 2008 R2 -> Click SQL Server Business Intelligence Development Studio.

Step 2: Start Analysis Services Project. Click File -> New -> Project -> Business Intelligence Projects -> select Analysis Services Project -> Assign Project Name (cubepract4) ->Click OK.



Step 3: Creating New Data Source

3.1 In Solution Explorer, Right click on Data Source -> Click New Data Source

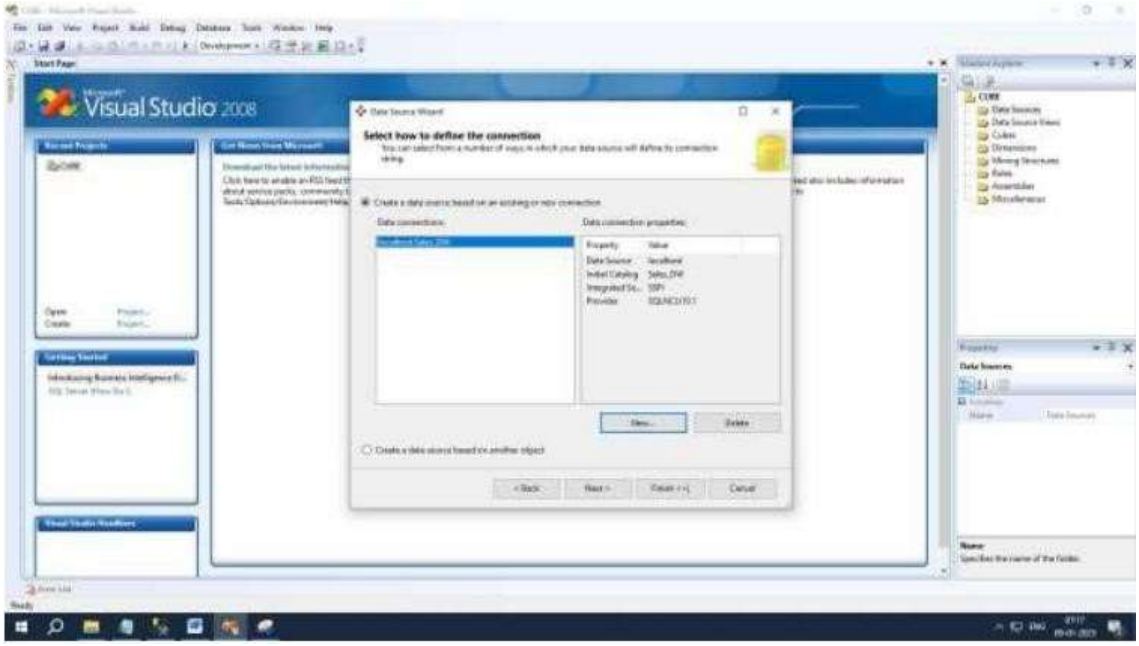


3.2 Click on Next

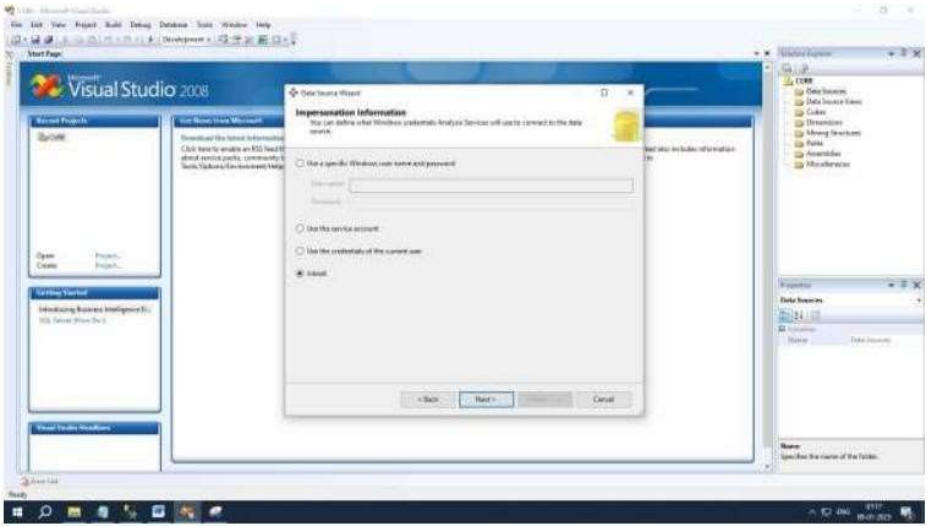
3.3 Click on New Button Creating New connection

1. Specify your SQL Server Name where your Data Warehouse was created
2. Select Radio Button according to your SQL Server Authentication mode
3. Specify your Credentials using which you can connect to your SQL Server
4. Select database Sales_DW.
5. Click on Test Connection and verify for its success.
6. Click OK

3.5 Select Connection created in Data Connections -> Click Next



3.6 Select option Inherit



3.7 Assign Data Source Name -> Click Finish

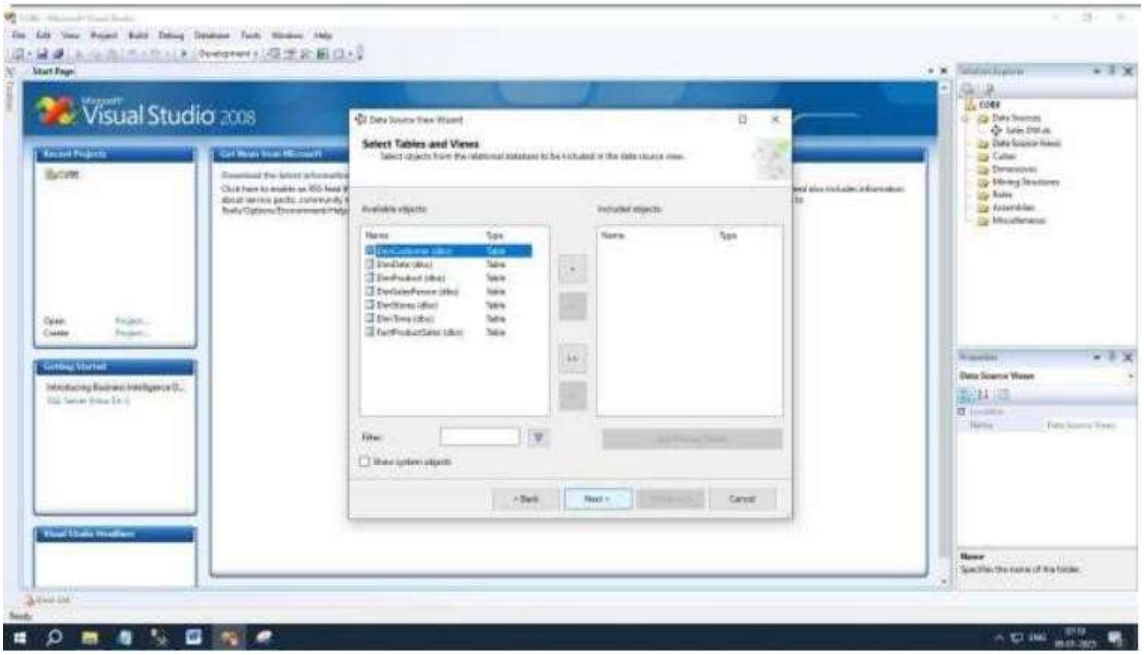
Step 4: Creating New Data Source View

4.1 In the Solution Explorer, Right Click on Data Source View -> Click on New Data Source View

4.2 Click Next

4.3 Select Relational Data Source we have created previously (Sales_DW) -> Click Next

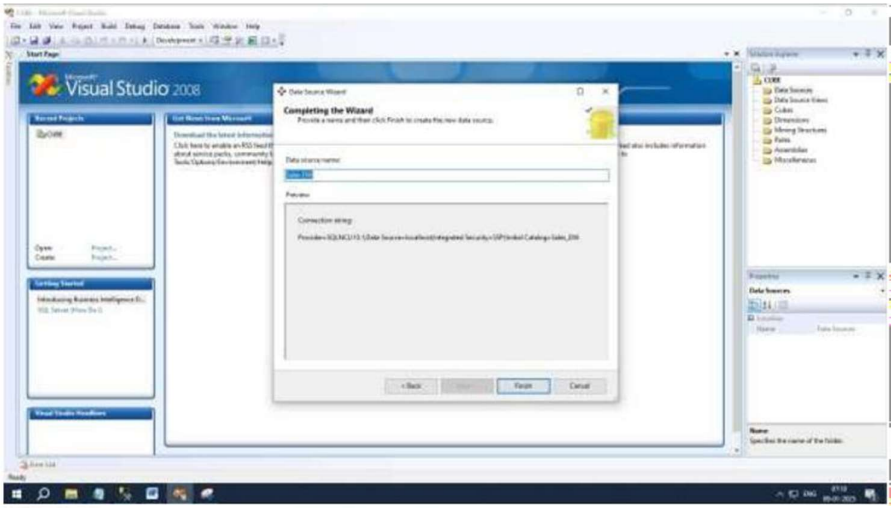
4.4 First move your Fact Table to the right side to include in object list.

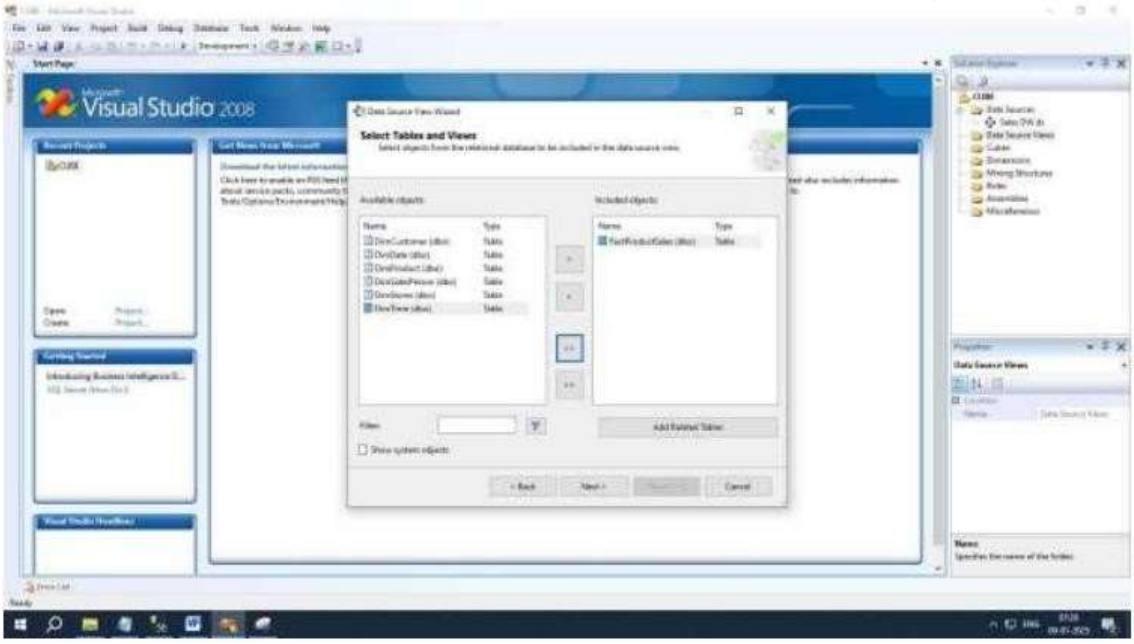


Select FactProductSales Table ->Click on Arrow Button to move the selected object to Right pane.

4.5 Now to add dimensions which are related to your Fact Table, follow the given steps:

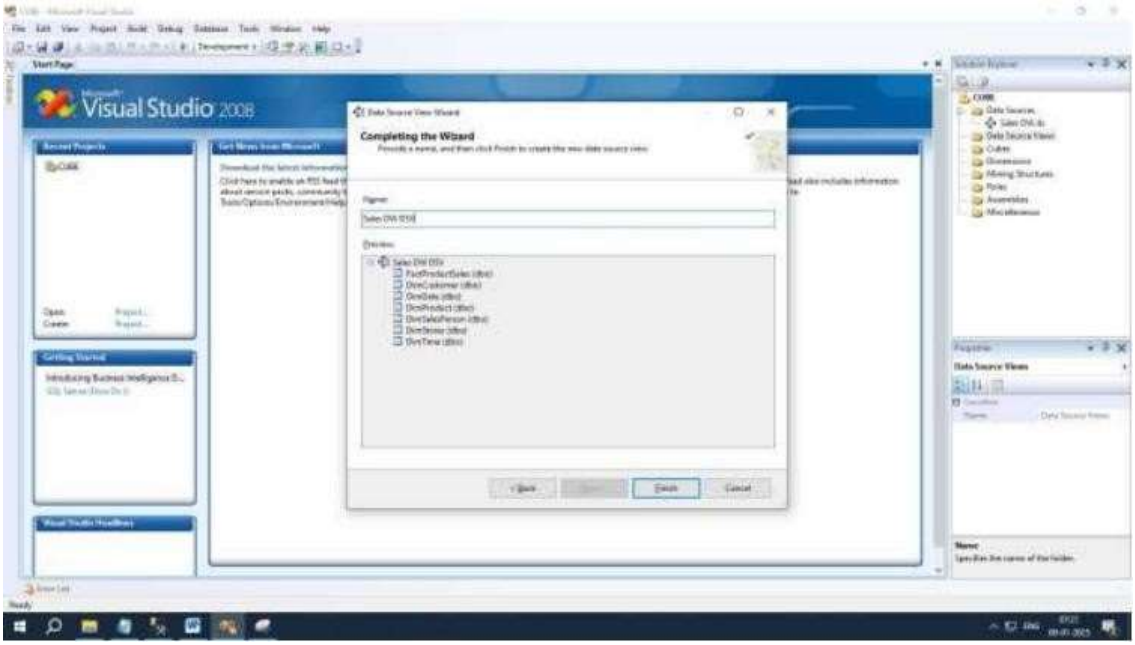
Select Fact Table in Right Pane (Fact product Sales) -> Click on Add Related Tables.



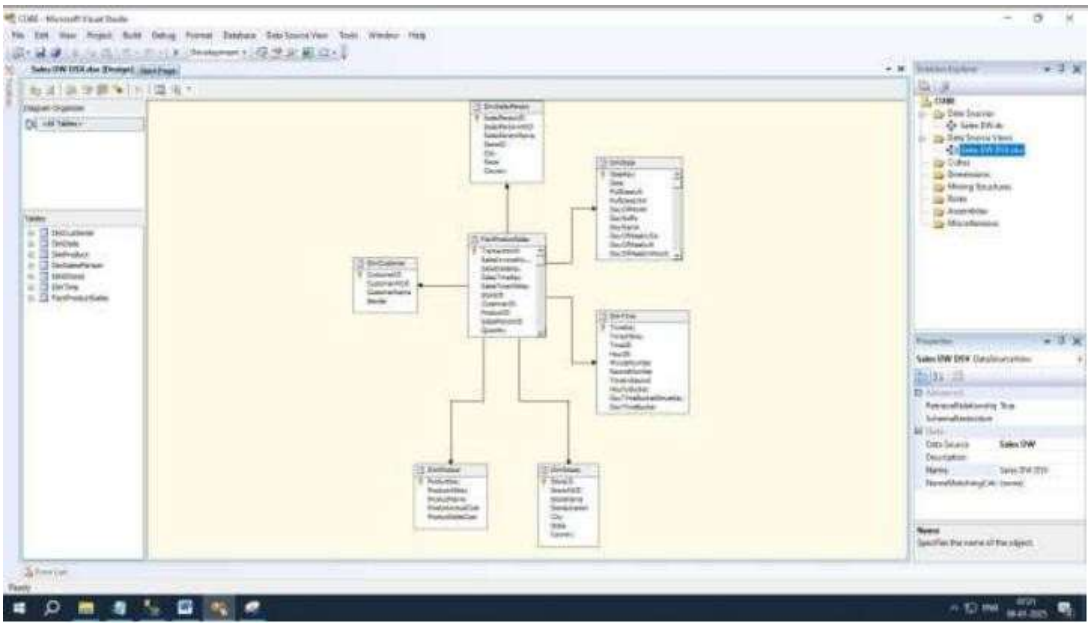


It will add all associated dimensions to your Fact table as per relationship specified in your SQL DW (Sales_DW). Click Next.

4.6 Assign Name (SalesDW DSV) -> Click Finish.



4.7 Now Data Source View is ready to use.

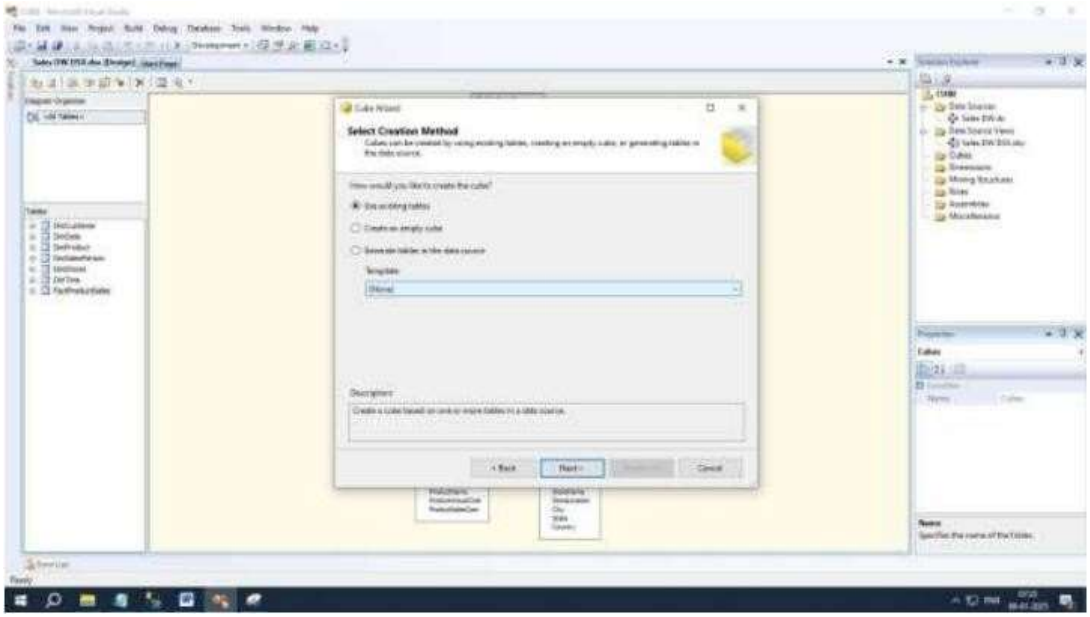


Step 5: Creating New Cube

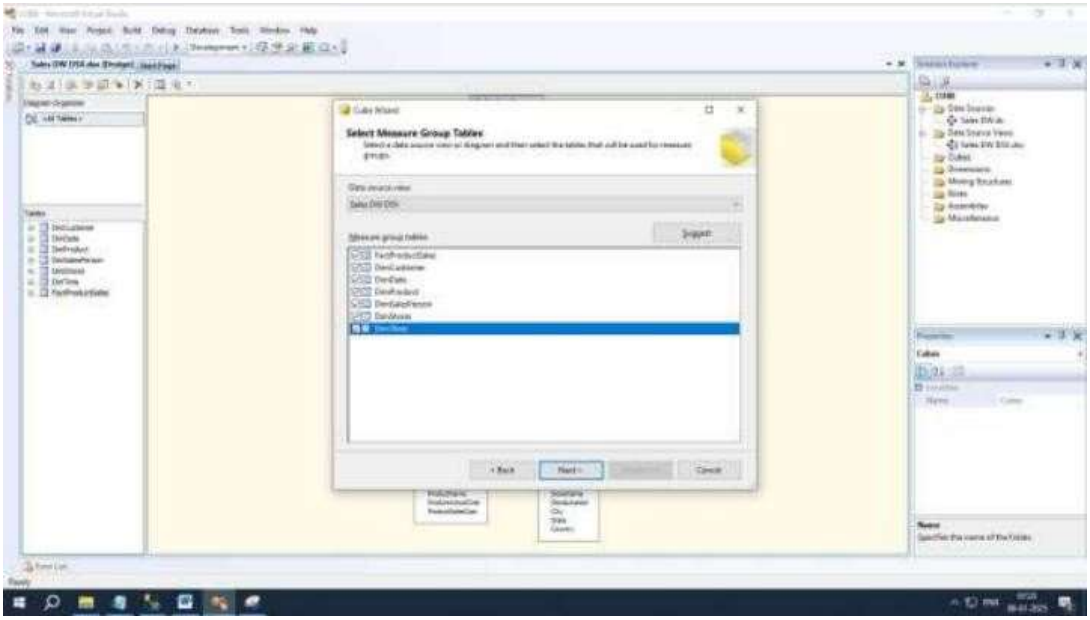
5.1 In Solution Explorer -> Right Click on Cube -> Click New Cube

5.2 Click Next

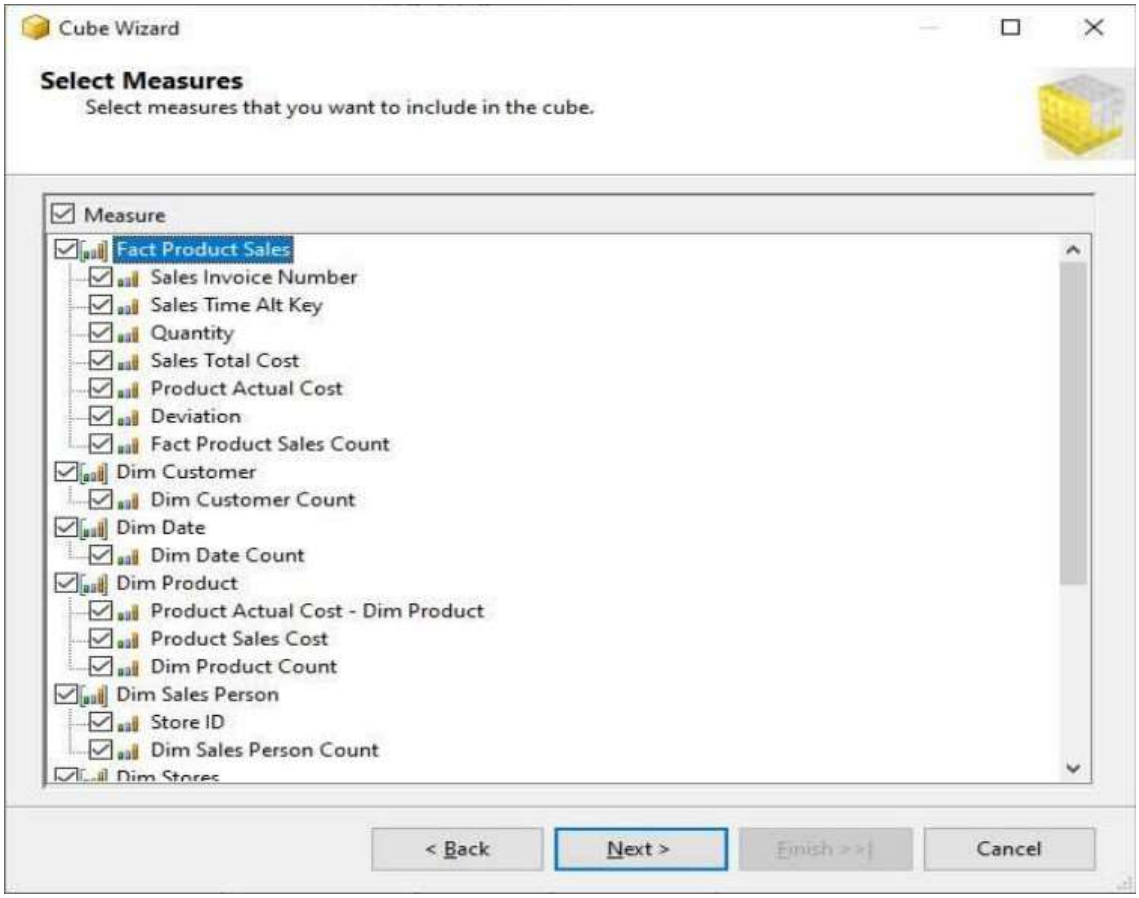
5.3 Select Option Use existing Tables -> Click Next



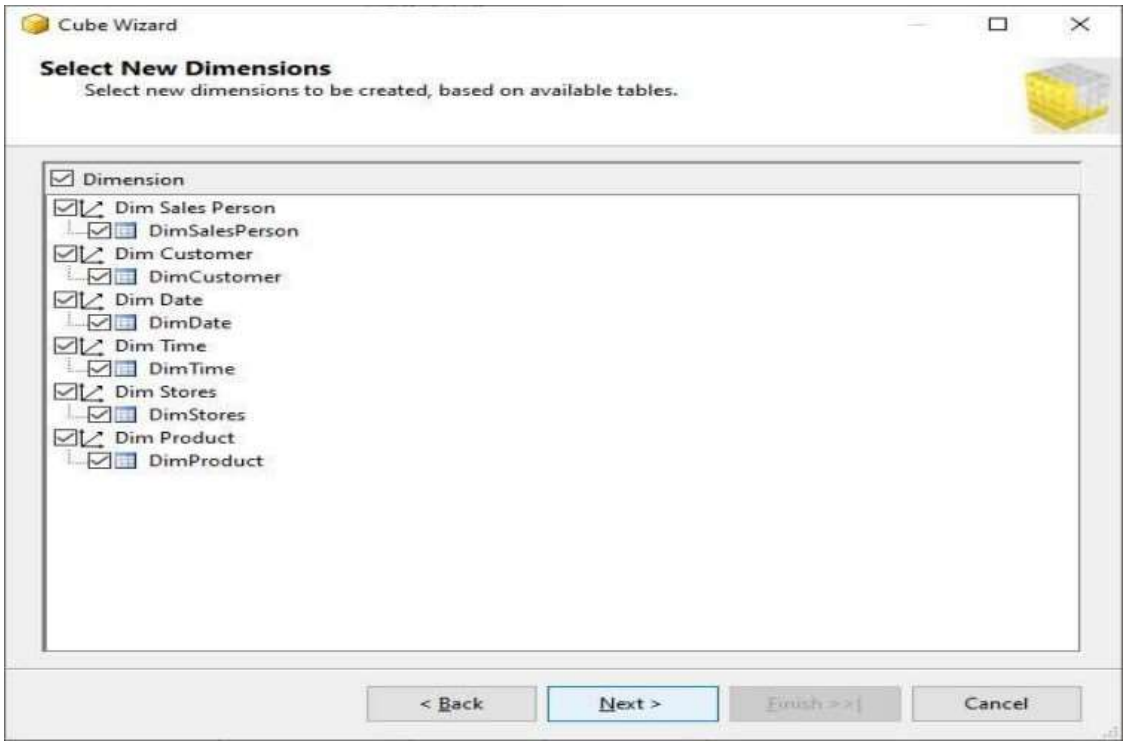
5.4 Select Fact table Name from Measure Group Tables (FactProductSales) -> Click Next.



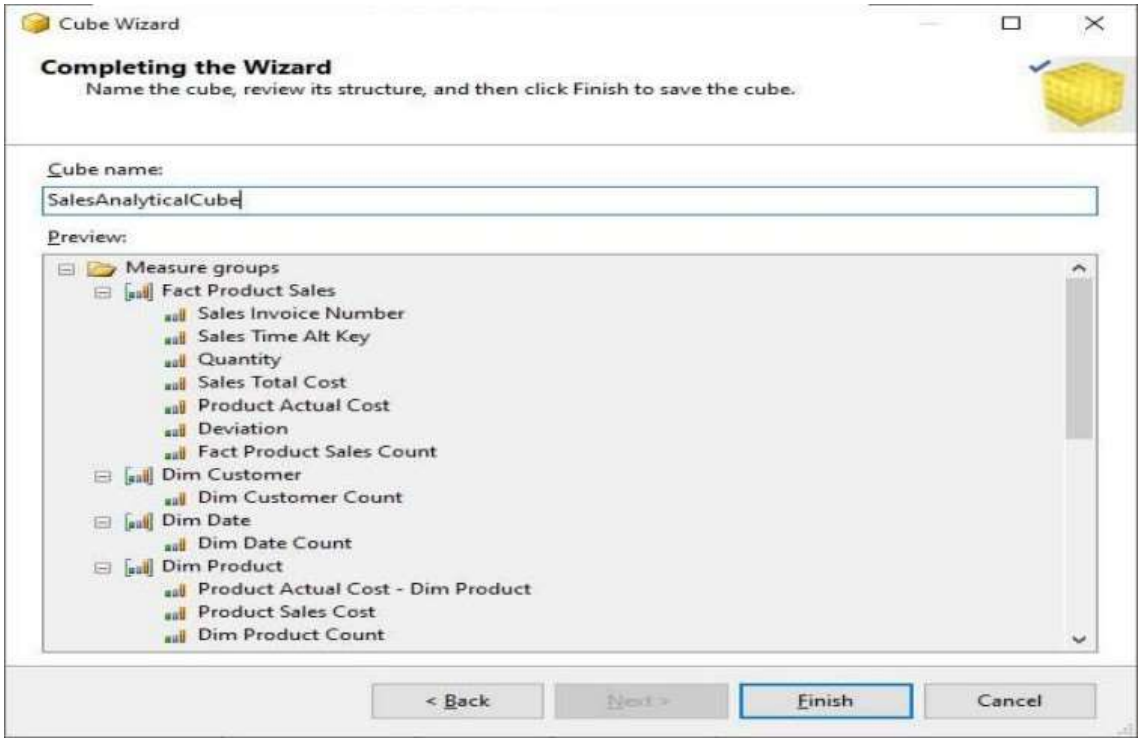
5.5 Choose Measures from the List which you want to place in your Cube-> Click Next



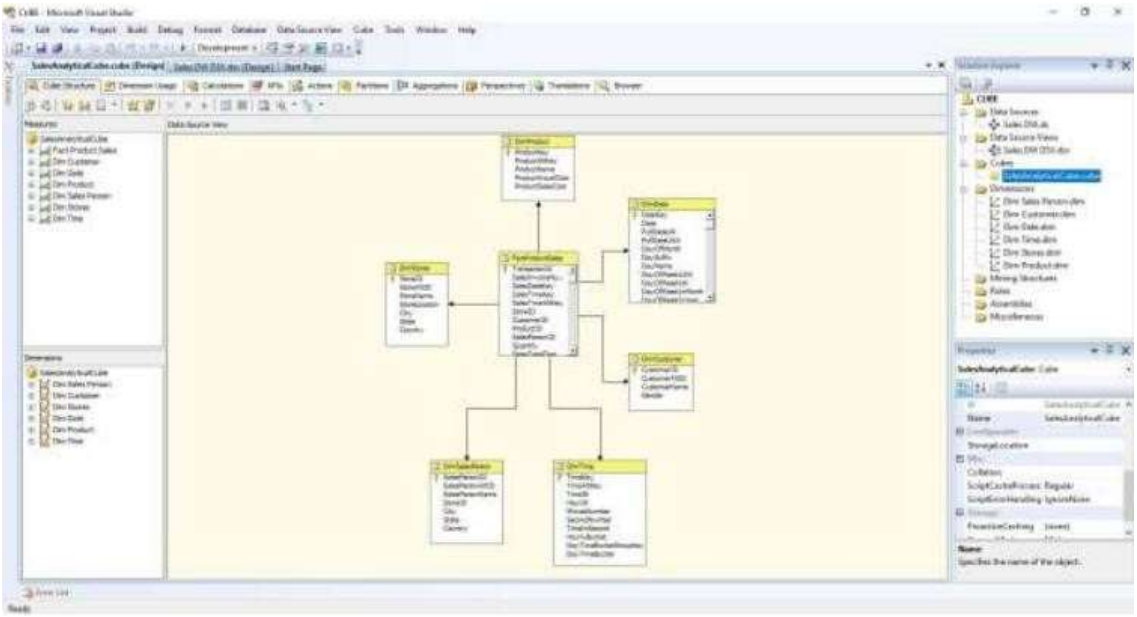
5.6 Select All Dimensions here which are associated with your Fact Table -> Click Next.



5.7 Assign Cube Name (SalesAnalyticalCube) -> Click Finish.

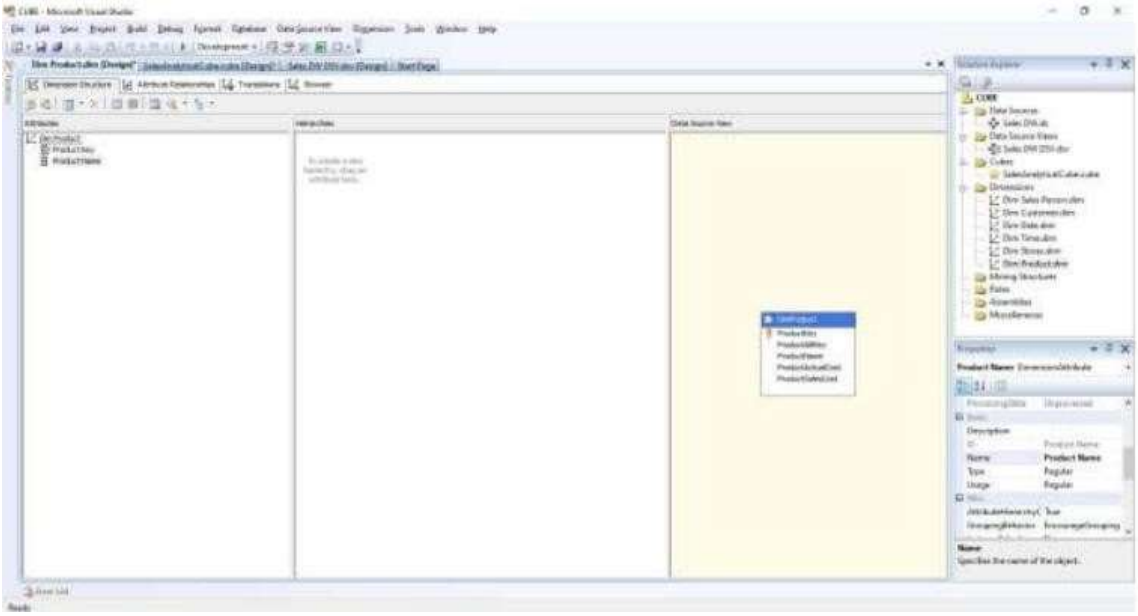


5.8 Now your Cube is ready, you can see the newly created cube and dimensions added in your solution explorer.



Step 6: Dimension Modification

In Solution Explorer, double click on dimension Dim Product -> Drag and Drop Product Name from Table in Data Source View and Add in Attribute Pane at left side.

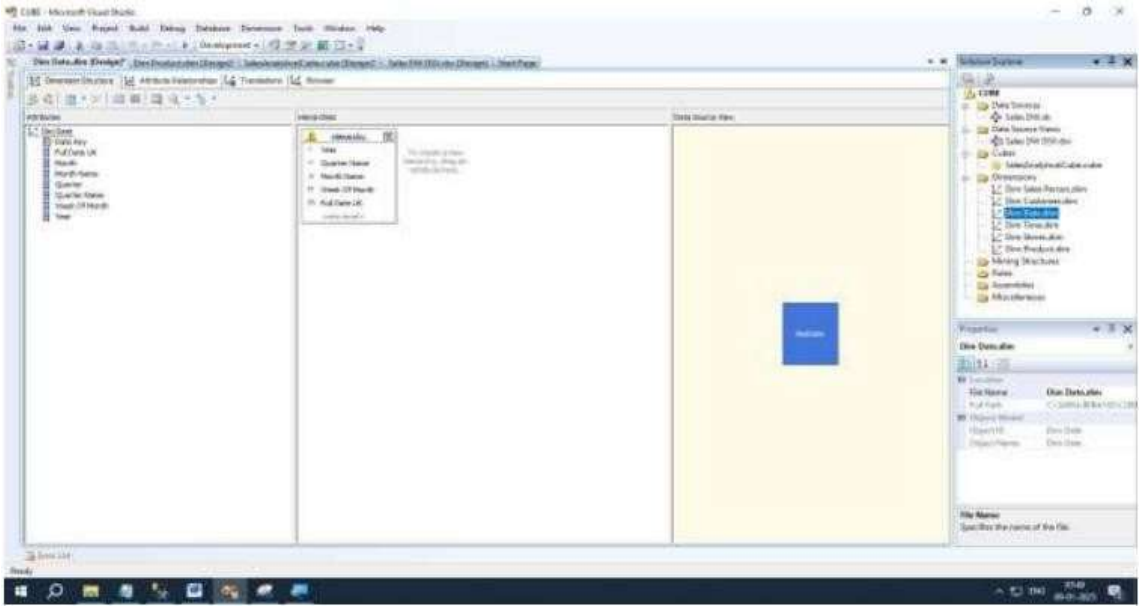


Step 7: Creating Attribute Hierarchy In Data Dimension

Double click on Dim Date dimension -> Drag and Drop Fields from table shown in Data

Source View to Attributes -> Drag and Drop attributes from leftmost pane of attributes to middle pane of Hierarchy.

Drag fields in sequence from Attributes to Hierarchy window (Year, Quarter Name, Month, Name, Week of the Month, Full Date UK).

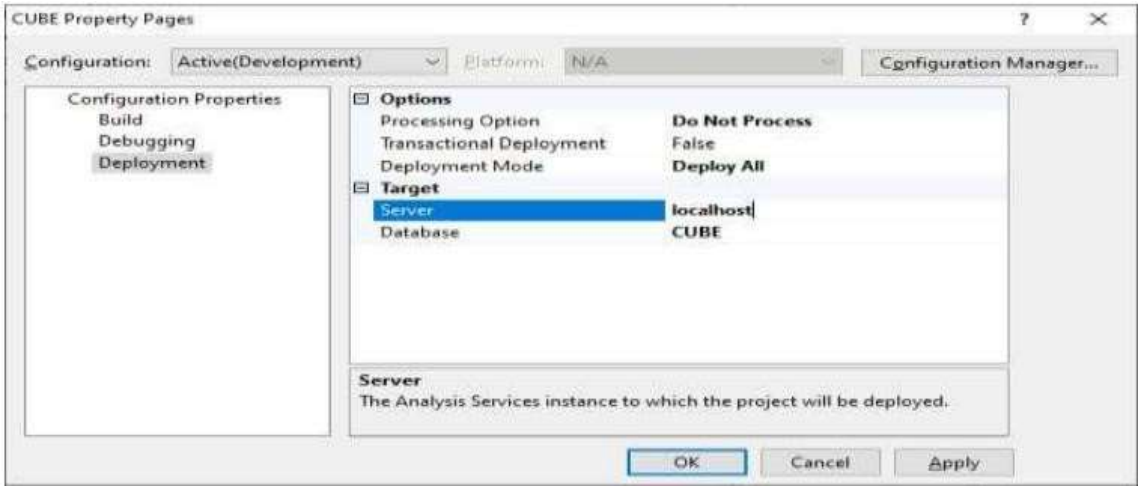


Step 8: Deploy the Cube

8.1 In Solution Explorer, right click on Project Name (cubepract4) --> Click Properties

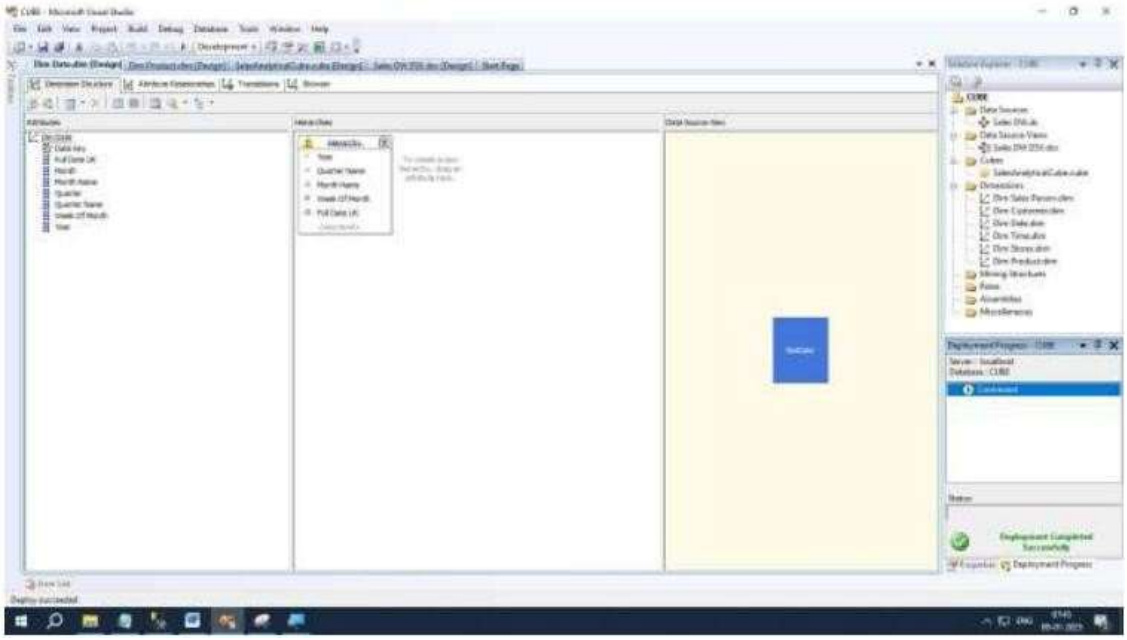
8.2 Set Deployment Properties First

In Configuration Properties, Select Deployment -> Assign Your SQL Server Instance Name Where Analysis Services Is Installed (localhost) (Machine Name/Instance Name) -> Choose Deployment Mode Deploy All as of now -> Select Processing Option Do Not Process -> Click OK.



8.3 In Solution Explorer, right click on Project Name (SalesDataAnalysis) --> Click Deploy.

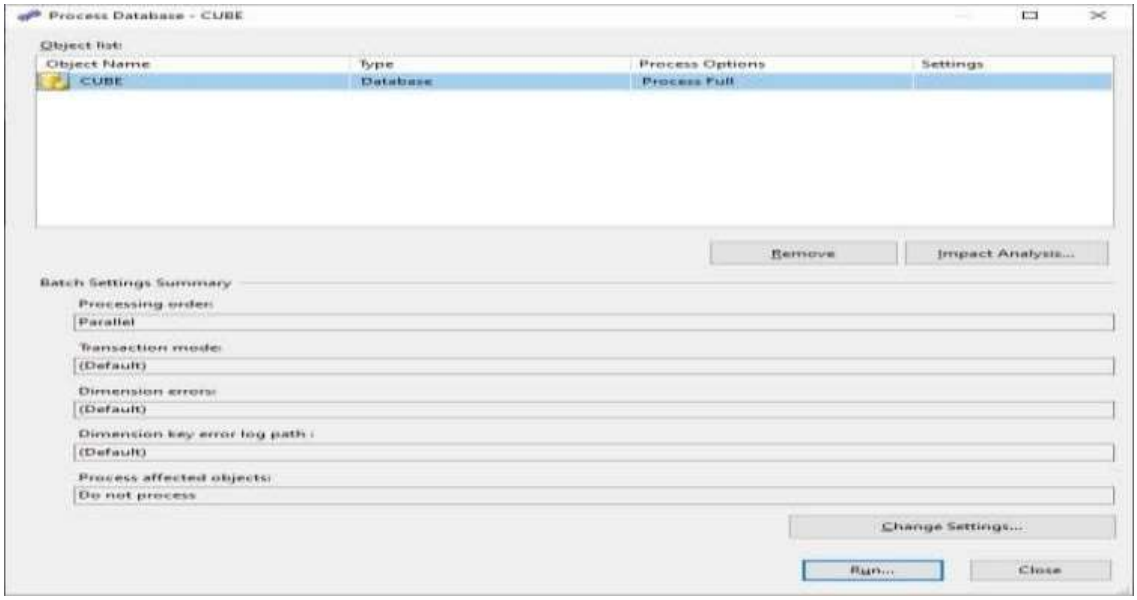
8.4 Once Deployment will finish, you can see the message Deployment Completed Successfully in Deployment Properties.



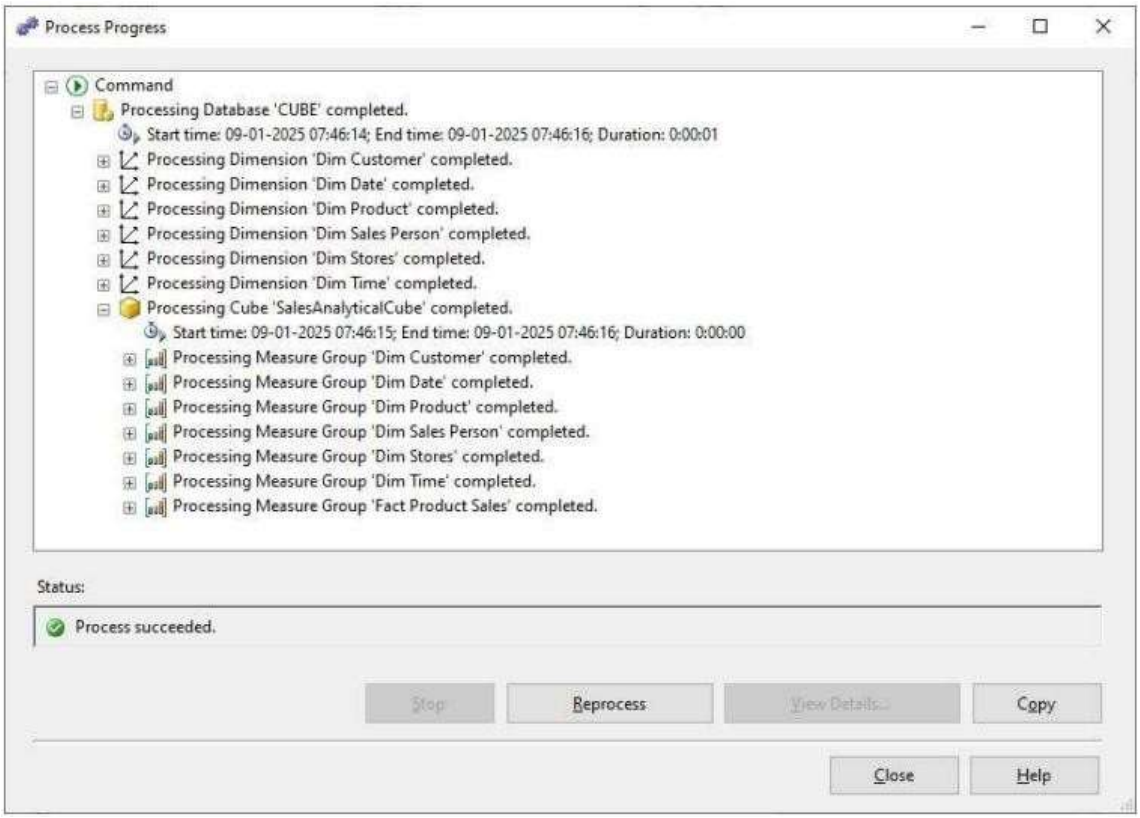
Step 9: Process the Cube

9.1 In Solution Explorer, right click on Project Name (SalesDataAnalysis) --> Click Process

9.2 Click on Run button to process the Cube.



9.3 Once processing is complete, you can see Status as Process Succeeded --> Click close both the windows for processing one after the other.



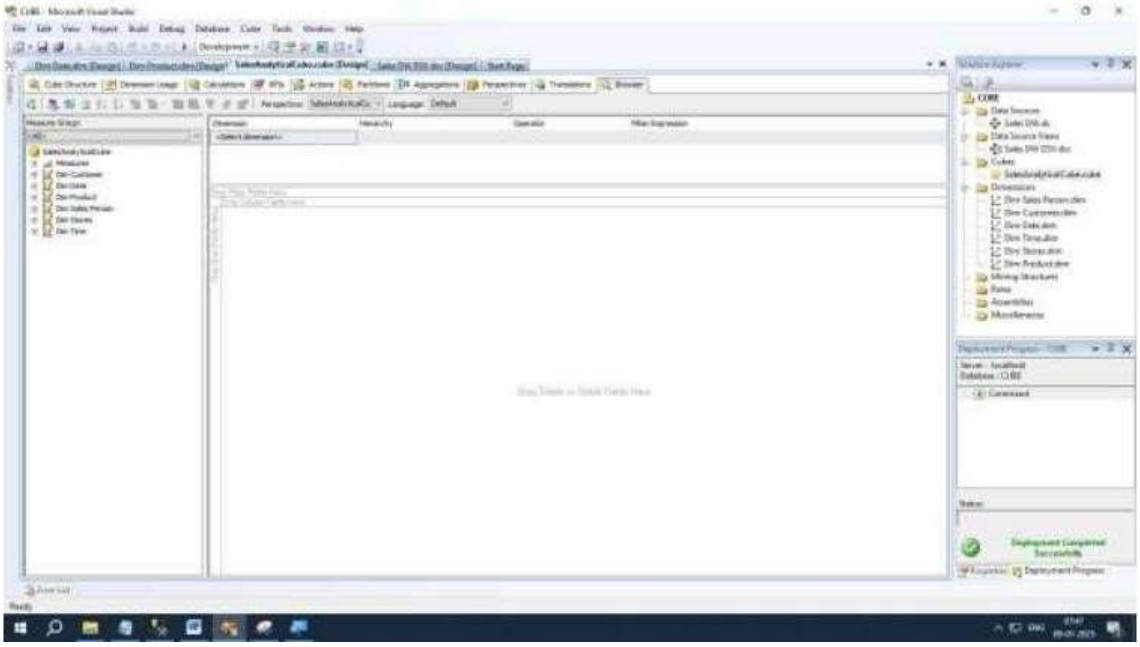
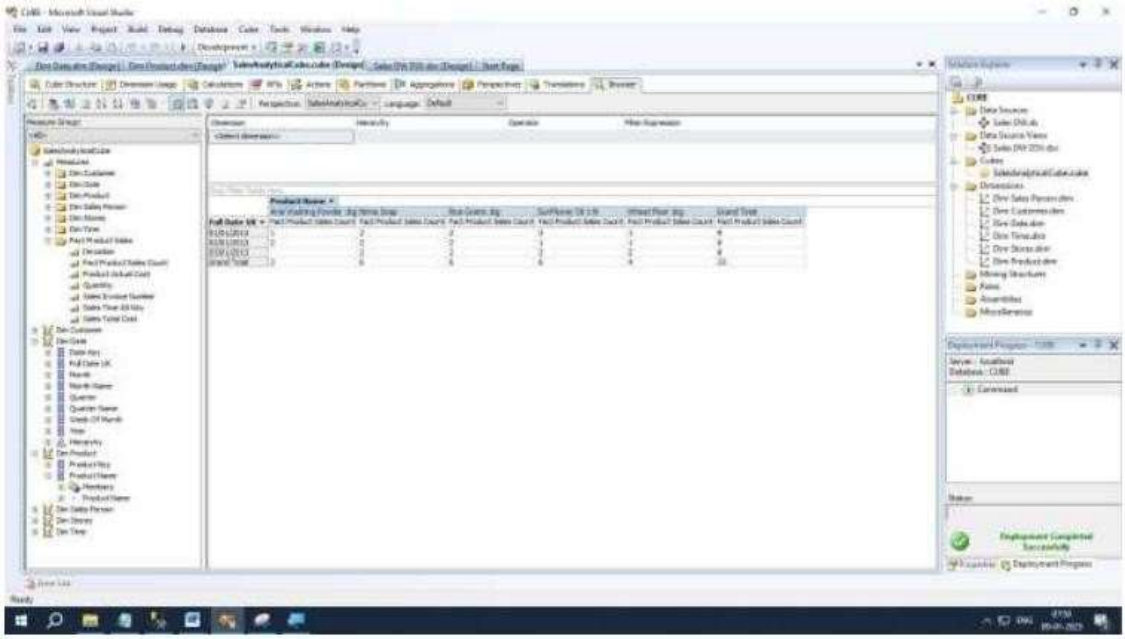
Step 10: Browse the Cube for Analysis

10.1 In Solution Explorer, right click on Cube Name (SalesAnalyticalCube) --> Click Browse.

10.2 Drag and drop measures in to Detail Fields, & Drag and drop Dimension Attributes in Row Field or Column fields.

Now to Browse Our Cube

1. Product Name Drag & Drop into Column

2. Full Date UK Drag & Drop into Row Field

3. FactProductSalesCount Drop this measure in Detail area